



Exploring the Relationship between Test Smells and Code Smells

Sandeep Reddy Munagala
Srinivasa Siddhartha Ratakonda

This thesis is submitted to the Faculty of Faculty at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Degree name. The thesis is equivalent to Weeks weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:

Author(s):

Sandeep Reddy Munagala

E-mail: samg22@student.bth.se

Srinivasa Siddhartha Ratakonda

E-mail: srst22@student.bth.se

University advisor:

Dr. Deepika Badampudi

Department of Software Engineering

Faculty of Faculty
Blekinge Institute of Technology
SE-371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background: In software evolution, code smells and test smells represent design flaws that can negatively impact the maintainability, readability, and reliability of both production and test code. These smells, although not immediately causing defects, can lead to long-term challenges such as increased technical debt, difficulty in debugging, and higher maintenance costs. This research focuses on the survivability of code and test smells, examining how long they persist in the codebase and their impact on software quality. Additionally, it explores the associations between code and test smells, providing insights into how the presence of one may indicate the other, and helping developers prioritise which smells to refactor based on their severity and longevity.

Objectives : The primary objective of this research is to find the survivability of Smells (Code Smells and Test Smells) and to find any possible associations between the smells code smells occurring in production code and test smells occurring in associated test code.

Methods: We conducted archival analysis on five repositories selected from GitHub. To find the survivability of smells we mine the selected repositories using the tool DesigniteJava to detect the smell and find their survivability. We also performed association rule mining to detect any associations with a minimum confidence of 50% between code smells and test smells.

Results: We found that test smells tend to survive longer than code smells. Among code smells, Feature Envy has the lowest survivability, while Magic Number exhibits the highest. For test smells, Ignored Test shows the lowest survivability, whereas Assertion Roulette has the highest. Additionally, we observed that the Long Statement smell in production code is frequently associated with Conditional Test Logic and Exceptional Handling in test code.

Conclusions: The findings highlight that test smells are often less prioritized for resolution compared to code smells, contributing to their higher survivability. The identified associations offer actionable insights for early detection and refactoring prioritization. These results provide a foundation for improving smell detection tools and practices, ultimately enhancing software quality and maintainability.

Keywords: Code Smells , Test smells , Software Repository Mining , Association Rule Mining.

Acknowledgments

We would like to express our sincere gratitude to our supervisor, Dr. Deepika Badampudi, for her unwavering support, invaluable guidance, and encouragement throughout the research and thesis writing journey. Her insightful feedback, expertise, and patience have played a crucial role in shaping this work. We feel incredibly privileged to have had the opportunity to learn from such a knowledgeable and inspiring mentor.

We are also deeply grateful to Mr. Tushar Sharma for his feedback on our thesis and also his knowledge and insights on the tool, Designite. Without which completing this thesis would have been very complex.

Finally, we would like to extend our heartfelt thanks to our family and friends whose constant support has been invaluable to us. Without their unwavering support, encouragement, and understanding completing this thesis would be very challenging.

Contents

Abstract	i
Acknowledgments	iii
1 Introduction	1
1.1 Background	2
1.1.1 Code Smells	2
1.1.2 Test Smells	3
1.1.3 Survivability	4
1.2 Scope	4
1.3 Outline	5
2 Related Work	7
2.1 Code Smells	7
2.2 Test Smells	8
2.3 Research Gap	9
3 Method	11
3.1 Aim and Objectives	11
3.2 Research Questions	11
3.3 Research Method	12
3.4 Study Design	12
4 Results and Analysis	21
4.1 Survivability of Smells (RQ1)	21
4.1.1 Survivability in Days	22
4.1.2 Survivability in Commits	23
4.1.3 Survival Curve	25
4.1.4 Code Smells	27
4.1.5 Test Smells	31
4.1.6 Summary of RQ1:	32
4.2 Association Rule Mining (RQ2)	32
4.2.1 Smells with High Support Value	33
4.2.2 Smells with High Confidence	33
4.2.3 Compound Smells with High Correlation	34
4.2.4 Summary of RQ2	34

5	Discussion	35
5.1	What Smells tend to survive more in Software projects	35
5.2	Are certain Test Smells more likely to associate with particular Code Smells	37
5.3	Threats to Validity	38
5.3.1	Construct Validity	38
5.3.2	Conclusion Validity	39
5.3.3	External Validity	39
5.4	Limitations	39
6	Conclusions and Future Work	41
6.1	Conclusion	41
6.2	Future Work	42
	References	45

List of Figures

3.1	Study Design	12
3.2	Designite Output	14
3.3	Code Smell Data	15
3.4	Test Smell Data	16
4.1	Median Survivability in Days	22
4.2	Median Survivability in Commits	22
4.3	Boxplot Distribution of Code Smells and Test Smells - Closed Interval	23
4.4	Survival Curve in Days and Commits for Code Smells and Test Smells	25
4.5	Survival Curve in Days	26
4.6	Survival Curve in Commits	27
4.7	Code Smells Count	27
4.8	Code Smell Survivability	28
4.9	Code Smell Survival Curve	29
4.10	Test Smell Count	30
4.11	Test Smell Survivability	30
4.12	Test Smell Survival Curve	31
4.13	Rule Mining Results	33

List of Tables

4.1	Project Summary	21
4.2	Number of Code and Test Artifacts Removed	22
4.3	Descriptive Statistics for the Survivability of Code and Test Smells, measured in Days and Commits,for each repository and the aggrega- tion of all repositories	24
4.4	Survival Probabilities for Code Smells and Test Smells across projects for both Days and Commits.	26
4.5	Code Smell Counts and Removal Percentages	28
4.6	Test Smell Counts and Removal Percentages	31

In software development, continual changes occur within the code base as software evolves. However, developers may not consistently adhere to quality standards when writing code due to time constraints or other factors. Consequently, this can result in the introduction of "code smells", which indicate potential flaws in software design [16]. These code smells, if not addressed, can significantly impact the maintainability of the software over time [57].

Code smells refer to code structures that violate design principles and hurt code quality. These issues can make systems difficult to understand and maintain. It is important to identify and address code smells in order to improve the overall quality of the codebase [47]. Fowler [16] defined various code smells, which refers to certain characteristics of code that signal a possible need for refactoring [16,56]. These characteristics may not necessarily indicate bugs, but they can make the software difficult to understand, modify, or extend due to a lack of clarity, simplicity, or modularity [56,58]. Examples of code smells include Duplicated Code, Large Classes, Large Methods, Inappropriate Intimacy between Classes, and Feature Envy, among others [23,31,34].

Just as code smells occur in production code, test smells occur in test code, the presence of test smells, introduced by Van Deursen et al. [53] can reduce the effectiveness of test cases in detecting bugs in production code [46]. These test smells are the indicators of poorly designed tests [53]. If a test code contains a flaw, it may lead developers to overlook hidden faults and hinder early detection of such faults [17]. Test smells negatively impact test readability and comprehension during maintenance activities [5,6,50]. Test code should be checked continuously to identify potential smells, and a refactoring mechanism must be used to remove them [48]. Tests containing smells are more prone to change and defects than tests that don't [39]. Also, production code is more likely to have defects when tested by smelly tests. Among the types of test smells, Indirect testing, Eager Test, and Assertion Roulette are associated with the highest change-proneness [46]. Moreover, the first two are also related to higher defect-proneness of the production code being tested [46].

Empirical research has shown that code smells limit comprehension [1], have high survivability [38,51] and files containing them are more prone to changes and faults [20,32]. Additionally, coexisting code smells can have a negative impact on maintainability [58]. Just as how above is true for code smells the same is also applicable to

test smells i.e., they also have high survivability [35,50], and are more prone to faults and changes [39,46,50]. With many similarities between code smell and test smell we don't know which one is more survivable or prone to changes or faults. Finding this will give us an understanding on how developers choose smells(code or test) to refactor. It is also necessary to know if the developers choose the smells(code or test) based on their severity or randomly. Studying the relationship between production code and test smells can assist in detecting those smells early [48]. Tufano [50] explored this by finding the relation between code smells and test smells; in their research, they concluded that test smells have high survivability and a relation exists between the code smells present in the production code and the test smells present in the production codes' associated test code. Tufano also concluded that knowing the relationships between the code smells and test smells can help in utilising the presence of code smell as an early indicator to identify the presence of test smells and vice versa. Finding any patterns between code smell and test smell can also help in assessing the severity level for the smells, which can help in prioritisation of smells for refactoring.

1.1 Background

In the world of software development, maintaining high-quality code is crucial for ensuring longevity, readability, and ease of maintenance of the codebase. However, due to tight deadlines, resource constraints, and lack of experience, code is often written hastily, leading to poorly structured and suboptimal solutions. These shortcomings can result in the introduction of smells—patterns that indicate deeper issues in the design or implementation of both production and test code. While these may not directly affect the code base immediately they can lead to long-term challenges such as increased technical debt, difficulty in debugging, and higher maintenance costs.

1.1.1 Code Smells

Code smells first introduced by Fowler in 1999 are considered design flaws and can be used as indicators of potential flaws in the design or implementation of production code [16]. Code smells are not bugs, but rather signs of deeper problems. The presence of code smells can hinder code comprehensibility [1], maintainability [58], and increase change and fault proneness [20] in production code. There are various types of code smells. Fowler defined 22 code smells in 1999 [16], but as more modern practices were introduced, many more code smells were defined [10,41]. In this study, we have selected the smells based on the availability of mining tools and the most recurring code smells as researched in [13]. The code smells that we selected are as follows: Abstract Function Call From Constructor, Feature Envy, Magic Number, Long Statement, Complex Method, Missing Default, Complex Conditional, Long Parameter List, Long Identifier, Long Method, and Empty Catch Clause. Given below are brief descriptions of the code smells we considered.

Feature Envy: This occurs when a function in one module spends more time communicating with functions or data inside another module than it does within its own module [16].

Magic Number: An unexplained number is used in an expression [42].

Long Statement: The code contains long statements typically those that do not fit in a screen [43].

Complex Method: A method with high cyclomatic complexity [42].

Missing Default: A switch statement does not contain a default case [42].

Complex Conditional: A complex conditional statement [42].

Long Parameter List: This code smell is described as a situation where a method or function requires too many parameters [16].

Long Identifier: An identifier with excessive length [42].

Long Method: A method is excessively long [42].

Empty Catch Clause: A catch block of an exception is empty [42].

1.1.2 Test Smells

Test smells are design flaws in test code that can lead to increased maintenance effort and decreased test reliability [24,46]. Studies show that smelly test code is more prone to failures and frequent modifications, hindering the testing process and reducing confidence in the codebase [46] and methods to refactor them are introduced [53]. Research has identified over 80 types of test smells, each contributing to various challenges in maintaining effective and reliable tests [8]. For this study, we selected test smells that are widely discussed and can be found using automated tools [2,37,45,52]. The Smells we selected are Assertion Roulette, Exceptional Handling, Conditional Test Logic, Constructor Initialization, Empty Test, Missing Assertion, Eager Test, Ignored Test. Given below are brief descriptions of the test smells we considered.

Assertion Roulette: This occurs when a test contains multiple assertions without clear messages or context, making it difficult to identify which assertion failed [53].

Exceptional Handling: This smell occurs when the passing or failing of a test method is explicitly dependent on the production method throwing an exception [37].

Conditional Test Logic: A test method that contains a conditional statement as a prerequisite to executing the test statement [36].

Constructor Initialization: A test class that contains a constructor [36].

Empty Test: A test method that is empty or does not have executable statements [36].

Eager test: A test method that calls several methods of the object to be tested [5].

Ignored Test: A test method that uses an '@ignore' annotation which prevents the test method from running [36].

1.1.3 Survivability

Survivability of smells can be defined as the duration or lifespan of a code or test smell from the time it first appears in the codebase until it is removed or refactored. It provides insights into how long these smells persist in a system, highlighting potential technical debt and maintainability challenges. Survivability can be calculated in number of days and number of commits. Number of days tells us how many days the smell survived before it was removed or refactored whereas number of commits tells us how many modifications are made to the codebase while the smell persists before it is removed or refactored.

1.2 Scope

This research aims to examine the relationship between code smells and test smells in software development. Specifically, the study will investigate the survivability of selected code smells and test smells over time across various software projects. The goal is to identify which types of smells—whether code smells or test smells—persist longer in the codebase, offering insights into which ones are prioritized or neglected during development. Survival analysis will be conducted on multiple software projects to determine whether trends are consistent across different projects. Additionally, the study will explore potential associations between code smells in production code and corresponding test smells in the related test code, providing a deeper understanding of how these issues may co-occur within software systems.

1.3 Outline

- **Chapter 1:** This section introduces the thesis providing the scope of the thesis this section also lays the foundations for key concepts relevant to this study, including code smells, test smells, survivability.
- **Chapter 2:** In this section we summarize the contributions made by different researchers in the area of code and test smell survivability and any associations between them.
- **Chapter 3:** This section provides with an overview of the research problem, aim, and research objective of this study and also the methodology and particular techniques used to carry out the research described in detail.
- **Chapter 4:** This section contains the results and analysis of survival analysis and association rule mining.
- **Chapter 5:** This section includes in-depth discussions of the results of the research and compares them with the findings of previous research.
- **Chapter 6:** This section summarizes the findings from the study and proposes possible future recommendations in this field.

2.1 Code Smells

Code smells arise from poor design decisions made by developers, impacting the software's maintainability [16]. Fowler [16] identified various types of code smells commonly found in software systems, including God Smell, Spaghetti Code, and Long Methods, totaling 23 distinct types of code smells. Since discovering code smell, the research community has thoroughly examined the concept from various angles.

Few studies have concentrated on methods for detecting various code smells. Travassos et al. [49] went about manually to identify smells, their study did not attempt to automate the process. Marinescu [27] presented a metric-based approach to detect code smells. Using textual analysis, Palomba et al. [35] presented TACO (Textual Analysis for Code Smell Detection), a method that uses a family of smells with varying degrees of granularity and nature. Apart from various detection strategies, researchers also presented many tools which help in detecting smells. Few tools are JSprit, Sonarqube, JDeodrent [38], DECOR [30], Designite [44]. Researchers used various proposed detection techniques and tools to aid themselves in finding the survivability of code smells, to understand the nature of code smells and also to find the impact of code smells on change and fault process.

Numerous studies have investigated the persistence of code smells in software systems. Peters et al. [38] explored the longevity of five specific code smells across seven open-source projects, finding that, on average, instances of code smells persisted through approximately half of the revisions examined. It was also observed that many developers were aware of the existence of code smells but didn't bother refactoring them. Tufano et al. [51] conducted a comprehensive analysis of over 200 open-source projects, revealing that code smells tend to appear when artifacts are created, with around 80% of these code smells persisting in the system. Moreover, a study focused on PHP code smells [41] indicate that such issues typically endure for about 37% of the application's lifespan, with variability depending on the size and complexity of the codebase. Larger codebases may exhibit a higher propensity for introducing and retaining code smells over time.

Code smells are also considered to be symptoms of anti-patterns and occur at the source code level [38]. Code smells often co-occur and certain combinations of code

smells can intensify maintenance issues and lead to unforeseen problems [18]. Kim Mens and Angela [23] found a strong correlation between Feature Envy and Long Methods while researching the possible relationships among four bad smells in open-source apps. Bad smells coexist and need to be removed to minimize code improvement efforts. The studies by Palomba [31, 34] found that Message Chains are the most common smell in classes with multiple design issues. The paper [14] found that Long Method and Divergent Change code smells co-occurring can also lead to architectural problems. When it comes to the source code of a system, the presence of a single anti-pattern doesn't significantly reduce its understandability as compared to the code without any anti-pattern. However, combining two or more anti-patterns in the same code negatively impacts its understandability and should be avoided during development and maintenance [1].

2.2 Test Smells

Van Deursen et al. [53] describes symptoms that could indicate potential design issues in test code, i.e., test code smells (or) test smells. These smells are different from those defined by Fowler for the production code. Van Deursen et al. also specified the possible issues that might occur due to the test smells, and the refactoring methods required to remove them. Test Smells presented in [53] have been further explored by Meszaro [29] where he states the reasons for why test smell appear. Few test smells include Mystery Guest, General Fixture, Lazy Test, etc., totalling 13 distinct types of test smells.

Test smells refer to poorly designed tests that can have a negative impact on the maintainability of both test suites and production code [5, 6, 50]. According to the research in [5], the presence of test smells can make test suites and production code less understandable. Test smells can be introduced when the corresponding test code is committed for the first time, and they often persist in a system over a prolonged period of time [50]. Test code should be checked continuously to identify potential smells, and a refactoring mechanism must be used to remove them [48]. Test-specific refactoring are proposed to remove these smells by Leon [53] and automated refactoring tool by [29]. The papers cited [53] and [54] focus on defining new test smells and [40], [7] designing detection techniques and tools to help indentify the test smells.

Researchers have employed various tools for detecting test smells. Tushar et al. [52] developed the tool that is able to detect both code smell and test smell. Luana et al. [28] utilized the JNose test tool to establish the TSSM dataset (Test Smells and Structural Metrics). Lambiase et al. [21] introduced the DARTS IntelliJ plugin, which implements a sophisticated detection mechanism to identify instances of three test smell which are General Fixture, Eager Test, and Lack of Cohesion of Test Methods at the commit level. Perumal et al. [36] developed the Tsdetect open-source tool to pinpoint 19 types of Java test smells. Aljedaani et al. [2] conducted a systematic mapping of test smell detection tools.

Tufano et al. [50] used TestHound tool to detect test smells like Assertion Roulette,

General Fixture, Eager Test, Mystery Guest and Sensitive Equality to find the survivability of test smells and found that after 1000 days, there is an 80% probability that a test smell has not been fixed yet. Even after 2000 days, the survival probability is still around 60%. This indicates that test smells tend to persist in a system for a long time.

Palomba et al. [33] analysed 110 open source projects, in order to investigate diffusion of test smells in automatically generated test cases. It was observed that 83% of the test classes were affected by at least one test smell where Assertion Roulette, Eager Test, and Test Code Duplication were the most frequent. It was also observed that all the test smells frequently co-occurs with Assertion Roulette, finally all test smells showed a positive correlation with structural characteristics of the systems. In a study conducted by Tahir et al. [48], a relationship was identified between specific properties of production code at the class level, and the occurrence of test smells. The study highlighted that metrics such as LCMO (Lack of Cohesion in Methods of Object), WMC (Weighted Methods per Class), and CC (Cyclomatic Complexity) derived from production code can serve as indicators of the presence of eager tests. However, it is noted that these same metrics are also commonly associated with code smells [15], further confirming the relationship between test smells and code smells. Tufano et al. [50] found that Eager Test smells are often associated with Class Data Should Be Private, Spaghetti Code, and Blob Class.

2.3 Research Gap

Many researchers have explored the co-occurrences of code smells with other code smells [14, 23, 34]. Additionally, some have investigated the co-occurrences between test smells [28, 33]. However, Tufano's study [50] addresses the co-occurrences of code smells with test smells, though it only considered five types of test smells. Tufano mentioned that test smells could indicate the presence of code smells in the tested classes and vice versa. While, Tahir et al. [48] checked whether complexity properties of a production class can be used as predictors of the presence of test smells in the associated unit test and found that WMC(Weighted Methods per Class), CC(Cyclomatic Complexity) and LCOM(Lack of Cohesion of Methods) of production code class are strong indicators of Eager Test and Duplicated Code Test Smells. Apart from above studies researchers have also found survivability individually for code smells and test smells having achieved same results like both having high survivability [35, 38, 50, 51]. There are currently no studies that have examined the survivability of code smells and test smells within the same system. This highlights a critical research gap, as it remains unclear which type of smell (code or test) impacts technical debt more.

Building on Tufano's findings, our research investigates the survivability of code smells and test smells and are there and are there any relations between code smells and test smells (i.e, whether a code smell is present when a test smell is present) and what developers choose between smells (code and test) to refactor. By doing so, we

could uncover patterns and correlations that assist in refining and enhancing code smell detection and prediction tools and refactoring practices. This advancement in knowledge could lead to practical outcomes, contributing to the gaps left by prior studies.

3.1 Aim and Objectives

This study aims to analyze the relationship between the survivability of code smells and test smells, and to investigate whether there are any associations between them.

The objectives of this study are as follows:

- **Objective 1:** Analyze the survivability of code smells and test smells over time in software projects.
- **Objective 2:** Investigate the co-occurrence of code smells in production code and corresponding test smells in related test code.

3.2 Research Questions

Based on the Aims and Objectives, the research questions were formulated.

1. **Which smells(code and test smells) tend to persist longer in software projects?**

Justification: Understanding the persistence of code and test smells can provide valuable insights into developer priorities and the evolution of software systems. By identifying which types of smells tend to linger, we can better understand the factors contributing to technical debt accumulation and software quality degradation. This knowledge can inform the development of effective strategies for smell detection, prevention, and remediation.

2. **Are certain test smells more likely to associate with particular code smells?**

Justification: We can uncover potential patterns by pinpointing associations between specific tests and code smells. These patterns can be used to develop targeted strategies to detect code smells. These can also be used to assess the accuracy of code smell detection tools. If certain test smells co-occur with specific, long-lasting code smells, it suggests that test smells can potentially act as early indicators of code quality issues. This knowledge can be used to develop better detection methods for these code smells, allowing developers to address them before they become more problematic.

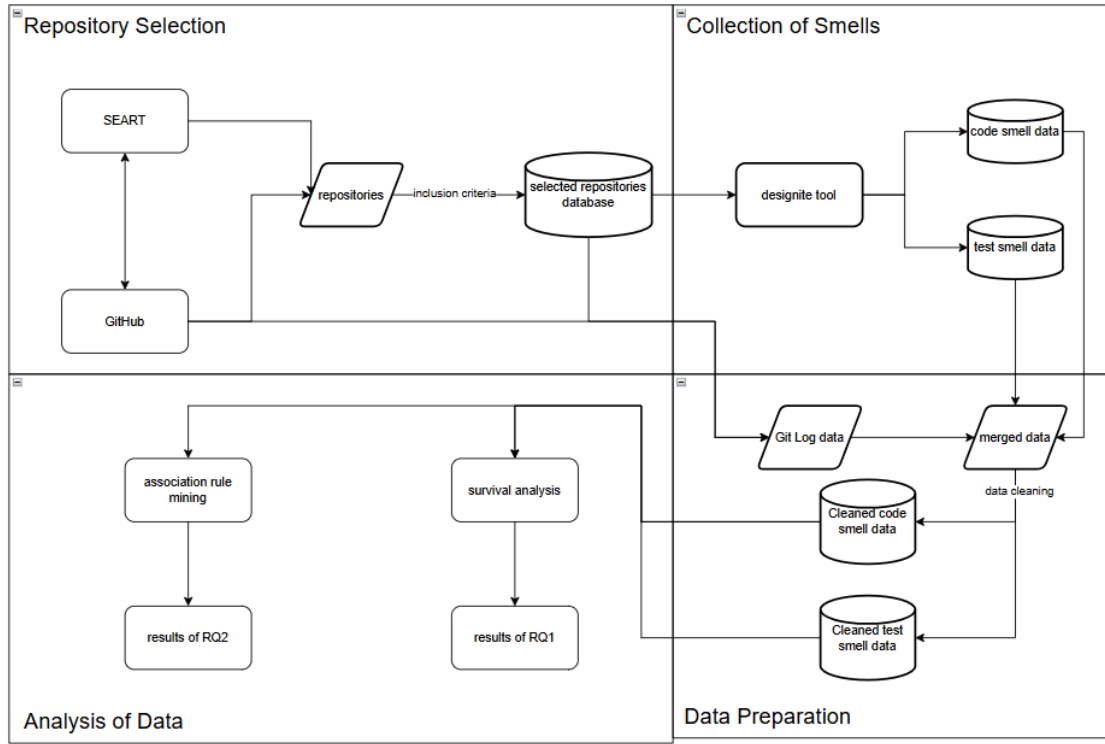


Figure 3.1: Study Design

3.3 Research Method

In this study, archival analysis was employed as the research method to analyze historical data from software repositories. This approach was chosen to examine the survivability of code, and test smells by tracking their presence across different commits. Additionally, it explored whether specific code smells co-occur with particular test smells throughout the development history, providing empirical evidence of these associations. The archival analysis is appropriate because it leverages real-world, historical commit data to uncover patterns related to software quality and maintenance [3, 9, 55].

3.4 Study Design

Our study design consists of the following steps:

1. **Repository Selection:** We used the SEART tool referenced by researchers to select the repositories from GitHub [8, 11]. These are the following criteria we used to select repositories for the study :
 - The repository must have a minimum of 1000 stars.
 - The repository should be a Java-based project.

- The repository should have at least 1000 commits and 100 KLOC.
- The repositories should be active and have a life span of over five years.
- The repository must have changes to test files.

There are millions of repositories on GitHub, but mining all the repositories is impossible given the resources and time frame available, therefore requiring the selection of a sample suitable for our study. To select the sample, we have utilized the above criteria. Selecting repositories having minimum of 1000 stars would indicate that the repository is recognized and valued by the community, implying a higher likelihood of quality code. Having a life span of over 5 years and consisting of minimum 1000 commits and 100 KLOC would suggest that the repository has a sufficient history of development activity. There is a need for changes in test file as our study focuses on both code smells and test smells persistence.

The above criteria were given to the SEART tool which is capable of automating repository selection by taking the inclusion criteria. The SEART tool retrieved over 100 projects when the criteria were applied; due to the computational limitations of the systems and the time constraints, we randomly selected five projects for mining the repository for code smells and test smells. The selection process was randomized by passing the repository names to a function, ensuring an unbiased and representative sample.

2. **Collection of Smells:** In this stage, we first used the `git log` command to extract detailed commit data, including commit IDs, timestamps, and author information, from the selected repositories [59]. This information allowed us to track changes made over time and provided a timeline for analysing the evolution of code and test smells. The commit data was foundational for linking each code change to the corresponding smells.

Selection of tool for collecting smells: In previous research, tools like JDeodorant, HistoryMiner, JSpirit, and Designite Java were used for detecting code smells, while JNose, TsDetect, and Designite Java were used for detecting test smells. However, most of these tools were outdated and no longer functioning properly.

As a result, we explored tools that are actively maintained and capable of accurately detecting both code and test smells. The selection criteria for these tools included their compatibility with modern software projects, accuracy in identifying various smell types, and ease of integration into our research workflow.

After evaluating available options, we opted for Designite Java due to its comprehensive detection capabilities, active development, and ability to analyze both code and test smells [44,52]. The tool also supports support multi-commit analysis, which is crucial for examining smells at the commit level. Additionally, its automation features allowed us to efficiently process large codebases

while ensuring reliable results. Despite its advantages, Designite Java has some limitations. While it supports multi-commit analysis and boasts a high precision rate of 99%, it has high computational requirements and can be time-consuming. Sometimes, the tool abruptly stops during mining, requiring manual restarting. Furthermore, in rare instances, it does not analyze all commits, leading to the omission of a few commits during the mining process.

Mining using Designite Java: DesigniteJava was chosen for its comprehensive detection capabilities and support for multi-commit analysis. After conducting the multi-commit analysis on the selected repositories, we obtained data organized in a directory structure. This directory contained folders named after their respective commits, and within each folder, we found CSV files detailing the smells identified at each commit.

The Figure 3.2 illustrates the structure of the data collected from Designite-Java.¹

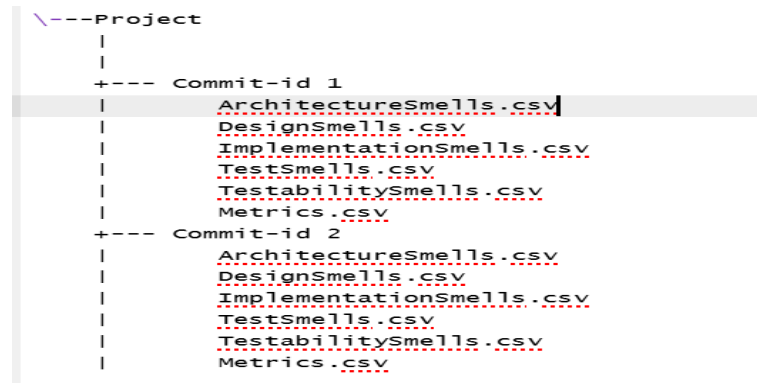


Figure 3.2: Designite Output

¹Designite java: <https://www.designite-tools.com/products-dj>

3. **Data Preparation:** In this step, we prepare the data for analysis. The raw data collected, as shown in Figure 3.2, cannot be directly used for analysis. We need to convert it into a format that allows us to extract meaningful insights. The data from Designite does not have information about the time when the commit was made. For this, we merged the `git log` data, which contained commit IDs and timestamps, with the DesigniteJava data, which captured code and test smells at each commit. This resulted in a unified dataset that mapped smells to specific commits, allowing us to track the timeline of smell introduction, persistence, and removal.

These are the steps followed in the data preparation for the above data generated.

- For each commit folder in the dataset, the folder name (representing the commit ID) was added as a new column in each CSV file.
- The next step is to merge files with the same name from different folders (commit IDs) into one combined file for each type of smell.
- In this step, the Git log data, which contains commit IDs and their corresponding timestamps, was merged with the preprocessed smell data. This process allows us to associate each detected smell with the time it was introduced in the repository.
- This is done for every project we selected.
- Finally, all project data was merged, including the ImplementationSmells.csv(Code Smell Data) and TestSmells.csv(Test Smell Data) files from all projects, into a single unified file for each type of smell(code smell and test smell).

Data after Data Preparation: After data preparation, we have two files: AllImplementationSmells.csv contains data for all identified code smells, while AllTestSmells.csv includes all detected test smells.

The dataset includes features such as Project Name, Package Name, Type Name, Method Name, Implementation Smell, Commit_time and Commit_id in the AllImplementationSmells.csv file. Similarly, the AllTestSmells.csv file contains the same features but replaces Implementation Smell with Test Smell. This can be seen in Figures 3.3 and 3.4.

From this data, we can say that there is a presence of smells in each file instance of a project at different commits. For each commit, the data indicates whether a specific file instance is affected by a particular smell.

Project Name	Package Name	Type Name	Method Name	Implementation Smell	Commit_id	Commit_time
maven	org.apache.maven	ReactorReader	hasBeenPacka	Long Statement	008639dec4ae4e29c	27-09-2010 22:18
maven	org.apache.maven	DefaultGraphBuilder	trimSelectedPr	Complex Method	008639dec4ae4e29c	01-03-2015 12:39
maven	org.apache.maven	DefaultMaven	setupWorkspac	Long Statement	009ec5dc4100c493c	12-02-2022 21:42
maven	org.apache.maven	DefaultModelValidatorTest	testMissingRep	Long Statement	00b902909460d0561	20-05-2013 23:10
maven	org.apache.maven	DefaultModelValidatorTest	testMissingRep	Magic Number	00b902909460d0561	20-05-2013 23:10
maven	org.apache.maven	DefaultModelValidatorTest	validateEffectiv	Long Statement	00b902909460d0561	21-05-2013 00:17
maven	org.apache.maven	TestRepositorySystem	buildArtifactRe	Long Statement	00d350eacae17df681	02-10-2022 10:41

Figure 3.3: Code Smell Data

Project Name	Package Name	Type Name	Method Name	Test Smell	Commit_id	Commit_time
maven	org.apache.maven	PathAssemblerT	distributionDirWit	Assertion roulette	05352442eb93be2bb460a	23-05-2020 09:55
maven	org.apache.maven	PathAssemblerT	distributionDirWit	Assertion roulette	05352442eb93be2bb460a	23-05-2020 09:55
maven	org.apache.maven	PathAssemblerT	distributionDirWit	Exceptional handling	05352442eb93be2bb460a	23-05-2020 09:55
maven	org.apache.maven	WrapperExecuto	WrapperExecutor	Constructor initializa	05352442eb93be2bb460a	23-05-2020 09:55
maven	org.apache.maven	StringSearchMod	locationTrackerSt	Assertion roulette	1ca65c79fa4e25c4b7da0	23-05-2020 10:07
maven	org.apache.maven	StringSearchMod	testInterpolateOb	Assertion roulette	1ca65c79fa4e25c4b7da0	23-05-2020 10:07
maven	org.apache.maven	StringSearchMod	testInterpolateOb	Assertion roulette	1ca65c79fa4e25c4b7da0	23-05-2020 10:07

Figure 3.4: Test Smell Data

4. Analysis:

RQ1: Which smells(code and test smells) tend to persist longer in software projects?

To answer this question we need to find when the smell(Code and Test) is introduced and when the smell(Code and Test) is removed from the system. There can be two types of commits to every smelly instance i .

- **Smell-Introducing Commit:** A commit $c(i)$ is classified as a Smell-Introducing Commit if an instance i in a file becomes affected by a smell as a result of the changes made in $c(i)$.
- **Smell-Removing Commit:** A commit $c(i)$ is classified as a Smell-Removing Commit if an instance i , previously affected by a smell, is no longer affected by the smell after the changes made in $c(i)$.

An instance may or may not have a Smell-Removing Commit as some smells are still unresolved i.e., they are still present in the latest commit at the time of mining. In such case, we assign the Latest Commit as the Smell-Removing Commit and call this kind instance as censored instances.

From this, we can categorise our data into two types:

- **Closed Data:** Data in which smells are removed and has a Smell-Removing Commit.
- **Censored Data:** Data in which smells are not removed, and the latest commit of the repository is considered as the "Smell-Removing Commit."

Now, the survivability can be calculated in terms of days and commits.

- **Days:** The # days between Smell-Introducing Commit and Smell-Removing Commit.
- **Commits:** The # commits between Smell-Introducing Commit and Smell-Removing Commit.

We used # days and # commits similar to Tufano et al. [51] and Zabardast et al. [59] to analyze code and test smell survivability. We consider both # day and # commits because in certain cases there may occur a period of inactivity in the project which leads to an unusual increase in the # days when considering survival of smells, to avoid this we consider both # days and # commits as a metric to calculate the survivability of smells. We only considered smells

that were removed due to refactoring and not due to code deletion. To ensure this, we checked the smell-removing commit with the git logs to see if any file containing the smell was deleted or not; if the file was deleted, we did not consider the smell as refactored. After doing this process for all the smells removed, we only considered those removed by any method other than code deletion.

We employed survival analysis to examine the survivability of the data. Survival analysis is a statistical technique used to analyse and interpret data concerning the time until a specific event occurs. In our case, the event of interest is the removal of code smells and test smells. Survival analysis provides the probability of the event occurring at a specific time. It uses the survival function, $S(t) = P(T > t)$, which indicates the probability that the smell, survives beyond a specific time t ; it considers both the closed data and censored data for calculating survival probability. Now it uses the survival model to create a survival graph using the data as survival probability as a function of time [51, 59]. The graphs' X-axis represents the units of measurement for time, days, and commits, depending on the context. The Y-axis represents the survival probability, indicating the likelihood that a particular code smell persists over time.

Now, we generate boxplots and survival curves to observe the survivability of the smells. Boxplots give an overall distribution of all the resolved smell instances, but there are cases where smells are still present at the latest commit, resulting in censored data. Survival curves are ideal in such scenarios as they take into account both observed removals and censored instances [50, 51, 59], resulting in a more accurate representation of code and test smell persistence over time.

RQ2: Are certain test smells more likely to associate with particular code smells?

To answer this question, we used Association Rule Mining, which is used to discover relationships or patterns between variables in large datasets. In our study, we check whether a particular code smell is present in the production code when a test smell is present in the production codes' associated test code.

As we have Code Smell Data and Test Smell Data separately in different files, as we can see in figures 3.3 and 3.4, we merged the data on code smells and test smells to establish relationships for association rule mining. The datasets consisted of code and test smell information, each associated with a unique Commit_ID and a TypeName field. 'TypeName' field corresponds to the name of the file in which the smell is present. To align these datasets, we utilised Commit_ID as a common key. We normalised the TypeName field from the test smells dataset by removing the "Test" suffix to match the corresponding TypeName in the code smells dataset. For instance, if the code smell data had a record with Commit_ID '101' and TypeName 'App', and the test smell data had a corresponding record with Commit_ID '101' and TypeName 'AppTest', these entries were matched and combined. The resulting merged dataset in-

cluded rows containing the Commit_ID, TypeName, code smells, and test smells for each matched pair. This process ensured that all relevant data was consolidated, allowing us to analyze the co-occurrence of code smells and test smells within the same commit. For example, in one instance, the merged dataset revealed that a God Class smell in the 'App' module was associated with an Assertion Roulette smell in the corresponding 'AppTest' module.

Our data now combines instances of production code affected by Code Smells and instances of test files exhibiting Test Smells within the same commit. We employed association rule mining algorithms, such as the Apriori algorithm [12], an unsupervised learning technique for pattern discovery. In our case, the patterns of interest are the co-occurrence of code smells and test smells.

This analysis helps us understand the relationships between code and test smells within a codebase. Here are some key metrics used in association rule mining:

Rules:

Association rules are expressed as:

$$A \rightarrow B$$

If A (antecedent), then B (consequent).

For example, "If Assertion Roulette is present, then God Class is also present."

Support:

Measures how frequently both the antecedent (A) and consequent (B) occur together in the dataset.

$$\text{Support}(A \rightarrow B) = \frac{\text{Occurrences of } A \cap B}{\text{Total instances}}$$

Example: A support value of 0.68 means 68% of the dataset contains both Assertion Roulette and God Class.

Confidence:

Measures the probability of the consequent (B) occurring given that the antecedent (A) is present.

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Occurrences of } A \cap B}{\text{Occurrences of } A}$$

Example: A confidence value of 0.76 means 76% of the time God Class appears when Assertion Roulette is present.

Lift:

Measures how much more likely A and B are to occur together compared to if they were independent.

$$\text{Lift}(A \rightarrow B) = \frac{\text{Confidence}(A \rightarrow B)}{\text{Support}(B)}$$

Lift > 1: Indicates a positive correlation between A and B (they occur together more often than expected by chance).

Lift = 1: Indicates no correlation between A and B.

Lift < 1: Indicates a negative correlation between A and B (they occur together less often than expected by chance).

We only considered associations where the confidence was greater than 0.5, as this threshold provided stronger correlations. By setting the confidence above 0.5, we ensured that the rules identified had a higher likelihood of being valid, meaning that when a specific test smell was present, there was a greater than 50% chance that a corresponding code smell would also appear. This approach helped us focus on the most reliable and meaningful associations, reducing the number of weak or trivial relationships. We also performed a chi-square test on the rules generated to see whether they were statistically significant.

4.1 Survivability of Smells (RQ1)

In this section, we show the results of our analysis. We mainly focused on the survivability of code smells and test smells. To answer our research question, we have analysed the survivability of code and test smells in 5 open source repositories (Maven, Mongo-java-driver, Spring, Seatunnel, and Zookeeper). Table 4.1 summarises the details for the selected repositories. Figures 4.1 and 4.2 presents boxplots for code smells (left) and test smells (right) in days and commits respectively. The graphs are presented in a log scale to accommodate the wide range of survivability values, providing better data visualisation and comparison. In Figure 4.3, the box plots are organized into two rows and five columns for easier understanding. Each column corresponds to one of the five open-source repositories we analyzed. Each further contains boxplots for both code smells (left) and test smells (right), allowing for a side-by-side comparison. The upper half of the graph represents the survivability of smells in days, while the lower half represents the survivability of smells in commits.

Table 4.3 provides the descriptive statistics of the survivability of code and test smells, measured in days and commits, for each repository and the aggregated data across all repositories. We used the median as a reference for our analysis as it minimises the effect of outliers [22], ensuring a more robust interpretation of the data. It is important to note that Table 4.3 and Figures 4.1 and 4.2 focus solely on smells that have been removed from the system (Closed intervals), offering insights into their persistence within the codebase. Table 4.2 shows that about 47% of code smells and 45% of test smells are removed from code deletion. The remaining are removed from either refactoring, feature development, major restructuring, etc.,

Project	# Commits	Stars	# Contributors	Creation Date	LoC (kLoC)	# Code Smells	# Test Smells
Maven	15,128	4,401	197	21-05-2009	296	6,858	900
Mongo Java Driver	8,496	2,619	170	15-01-2009	603	14,803	4,038
Spring Batch	6,499	2,746	192	17-11-2010	212	8,657	4,374
Seatunnel	4,508	8,128	316	05-08-2017	419	9,442	1,667
Zookeeper	2,603	12,283	227	27-03-2009	191	4,733	1,705

Table 4.1: Project Summary

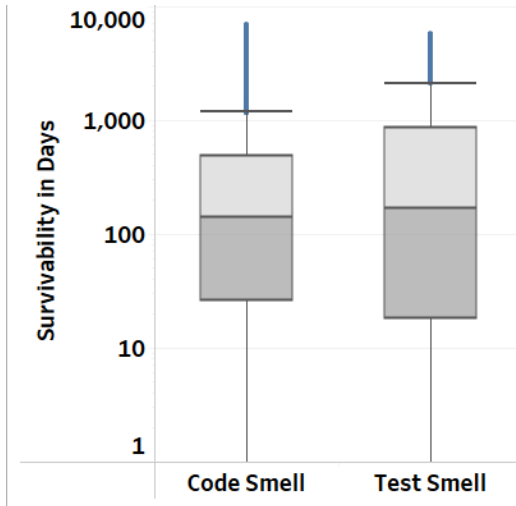


Figure 4.1: Median Survivability in Days

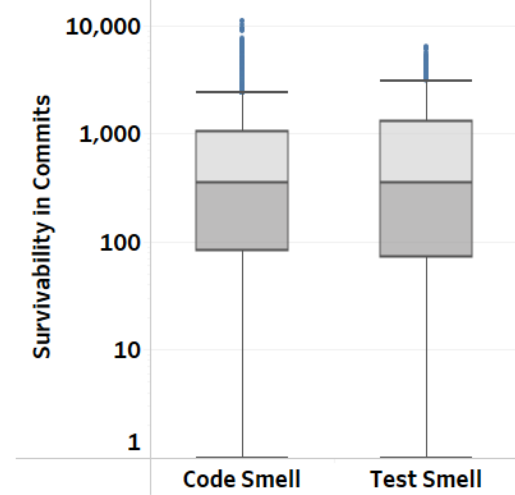


Figure 4.2: Median Survivability in Commits

Type	Removed by code deletion	Removed for Other Reasons
Code	47%	53%
Test	45%	55%

Table 4.2: Number of Code and Test Artifacts Removed

4.1.1 Survivability in Days

From Figure 4.1 and Table 4.3, we observe that, in general, test smells tend to survive slightly longer than code smell instances, with test smells persisting for up to 170 days compared to 140 days for code smells. This trend highlights the relative persistence of test smells in software systems. However, variations arise when analyzing individual repositories.

- From Figure 4.3, we observe that in Maven, Spring-Batch, and Zookeeper, test smells consistently survive longer than code smells, aligning with the overall trend.
- On the other hand, in Mongo Java Driver and Seatunnel, code smells exhibit longer survivability than test smells, deviating from the general pattern.

To investigate this discrepancy, we referred to Table 4.1, which provides detailed information about each repository. The table reveals that Mongo Java Driver and Seatunnel have significantly larger codebases, with 603 KLOC and 419 KLOC, respectively, compared to the other repositories. This suggests that codebase size plays a significant role in the survivability of smells. Larger codebases are inherently more complex, which may delay identifying and addressing code smells, prolonging their presence. Additionally, these repositories have more code smells (14,803 and 9,442, respectively), which may contribute to their increased survivability. These findings emphasise the importance of considering project-specific characteristics, such as codebase size and the volume of smells when analysing and addressing smell persistence.

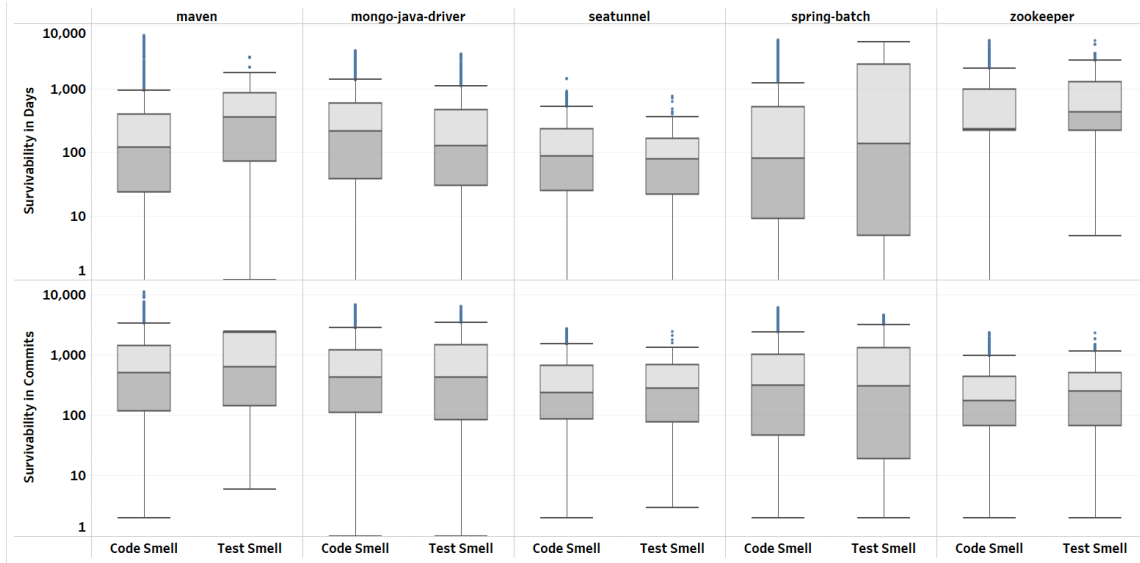


Figure 4.3: Boxplot Distribution of Code Smells and Test Smells - Closed Interval

Tailored strategies are essential to manage and refactor smells in large and complex repositories effectively.

4.1.2 Survivability in Commits

From Figure 4.2 and Table 4.3, we observe that the survivability of code smells instances and test smells in terms of commits is relatively similar. For the aggregated data, the median survivability is 350 commits for code smells and 348 commits for test smells. However, minor differences can be seen when analysing individual repositories.

- In the Maven project, test smells have a higher median survivability (635 commits) compared to code smells (504 commits).
- In the Mongo Java Driver repository, the median survivability for code smells is 431 commits, whereas test smells have a median of 427 commits, showing very little difference.
- Spring Batch is also similar to Mongo-java-driver in commit survivability i.e., it also has a small difference in the median survivabilities for test smells (309 commits) and code smells (311 commits).
- For Seatunnel and Zookeeper, the median survivability of test and code smells is quite similar, with very small differences (e.g., 235 commits for code smells vs. 281 commits for test smells in Seatunnel).

Project	Min	1st quad	median	mean	3rd quad	Max	Smell
Survivability							
maven	0	24	122	460.13	396.5	952	code
maven	1	73	361	529.52	858	1797	test
mongo	0	39	213.5	461.98	590	1415	code
mongo	0	30	127	394.94	468	1120	test
spring	0	9	80	823.4	514	1267	code
spring	0	5	138	1273.1	2461	5633	test
seatunnel	0	25	87	181.48	233	538	code
seatunnel	0	22	79	110.36	165	364	test
zookeeper	0	223	234	769.5	991	2139	code
zookeeper	5	223	428	896.1	1273	2839	test
all	0	26	140	531	491	1188	code
all	0	18	170	756.9	859	2120	test
Commit survivability							
maven	2	118	504	1051	1446	3434	code
maven	6	145	635	1140	2395	2494	test
mongo	1	110	431	870	1219	2877	code
mongo	1	84	427	963	1455	3505	test
spring	2	47	311	752	1015	2453	code
spring	2	19	309	867	1302	3218	test
seatunnel	2	87	235	449	676	1550	code
seatunnel	3	77	281	398	681	1363	test
zookeeper	2	68	173	344	436	983	code
zookeeper	2	68	253	384	513	1173	test
all	1	84	350	783	1037	2462	code
all	1	72	348	859	1298	3132	test

Table 4.3: Descriptive Statistics for the Survivability of Code and Test Smells, measured in Days and Commits, for each repository and the aggregation of all repositories

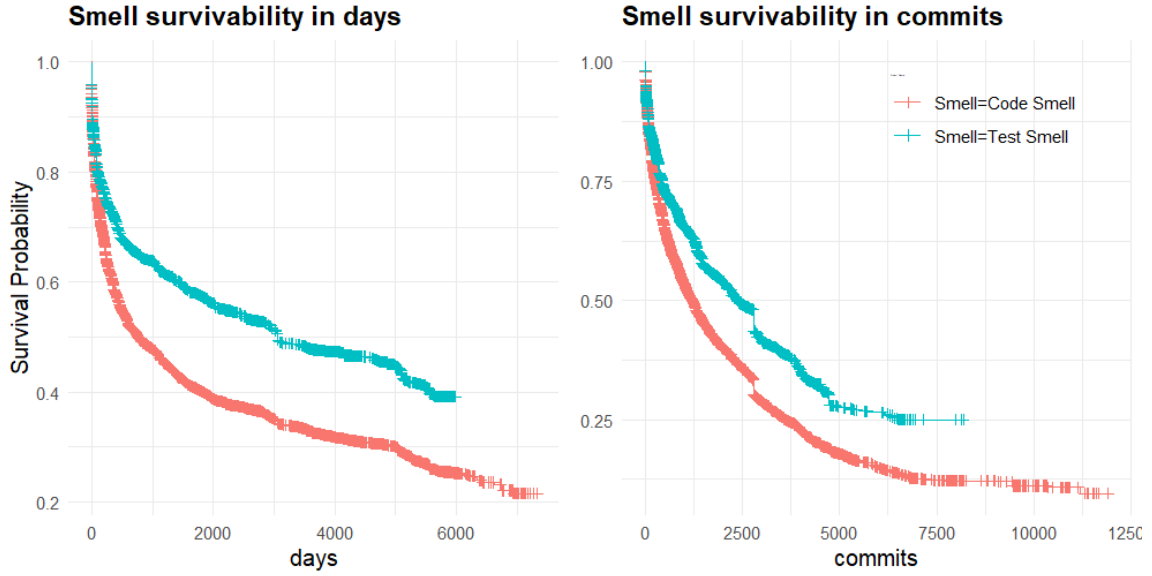


Figure 4.4: Survival Curve in Days and Commits for Code Smells and Test Smells

4.1.3 Survival Curve

In addition to using boxplots and descriptive statistics, we analyzed the survivability of code and test smells with survival curves. While boxplots offer a snapshot of survivability distribution of the instances affected by smells by highlighting medians, quartiles, variability, and outliers across groups, they do not account for the timing of events or censored data (e.g., unresolved smells). Survival curves, such as Kaplan-Meier plots, provide a time-based analysis, illustrating the probability of a smell persisting and its gradual decline as smells are removed, while also accommodating censored data. This approach offers a more detailed understanding of how long smells persist, including those yet to be resolved. Figure 4.4 represents the survival curves for code smells and test smells in terms of days(left) and commits(right). From this we see that Test smells have high survival than code smells. while the survival probability of code smells and tests are almost same at first 500 days and commits. Figure 4.5 and 4.6 represent the survival curves for code smells(left) and test smells(right) in terms of days and commits, respectively, for the repositories we selected. From Figure 4.5 we have observed :

- Survivability of smells, whether code smells or test smells, varies for different repositories. Survival curves for each project differ from one another due to various factors, including differences in project size, codebase structure, development practices, maintenance strategies, etc.
- The repository *zookeeper* has the highest probability for both code and test smells to survive.
- The repository *Mongo Java* exhibits the opposite trend, with a low probability of smells surviving.
- In the *maven* repository:

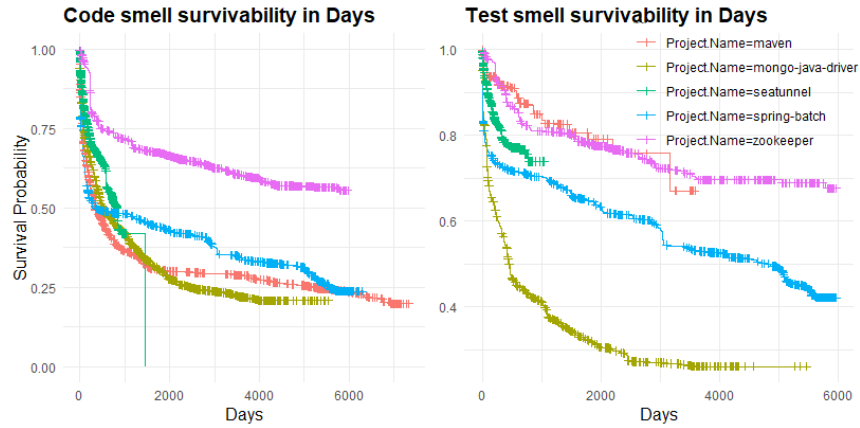


Figure 4.5: Survival Curve in Days

- Code smells have a low probability of surviving.
- Test smells, however, have a high probability of surviving.
- In the *spring batch* repository:
 - Code smells have a lower probability of surviving compared to other repositories during the first 500 days.
 - As the number of days increases, the survivability probability of code smells in *spring batch* surpasses that of other repositories.

To evaluate which smells are more likely to persist at a specific point in time, we used 500 days and 500 commits as reference points, following the methodology adopted in previous studies [51, 59]. Table 4.4 presents the survival probabilities of code smells and test smells at these reference points, measured in terms of both commits and days, across the selected repositories.

From Table 4.4, we observed that, except for the mongo-java-driver repository, the survival probability of test smells is higher than that of code smells in terms of both commits and days across all other repositories. This trend is evident at every point in time, as illustrated in Figure 4.5. From Table 4.4 we can see that the chance for code smell surviving 500 days and commits is about 54% and 64% respectively, and for test smells it is 67% and 72% respectively.

Project	Survival Probability (Days)		Survival Probability (Commits)	
	Code	Test	Code	Test
maven	0.4588125	0.9095422	0.6539591	0.9176467
mongo-java-driver	0.5045313	0.4665654	0.6018059	0.6061259
seatunnel	0.6448278	0.7709220	0.7375110	0.8688028
spring-batch	0.4909368	0.7176136	0.5957861	0.7212412
zookeeper	0.7484651	0.8682994	0.7037108	0.7956299
all	0.5426004	0.6688303	0.6428851	0.7190727

Table 4.4: Survival Probabilities for Code Smells and Test Smells across projects for both Days and Commits.

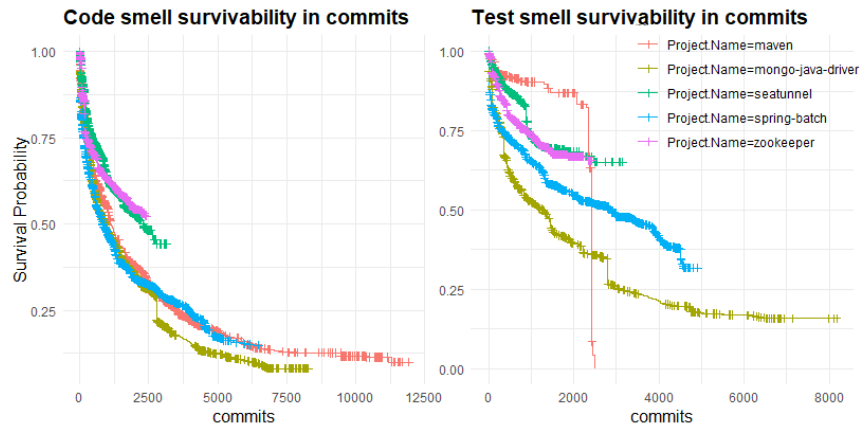


Figure 4.6: Survival Curve in Commits

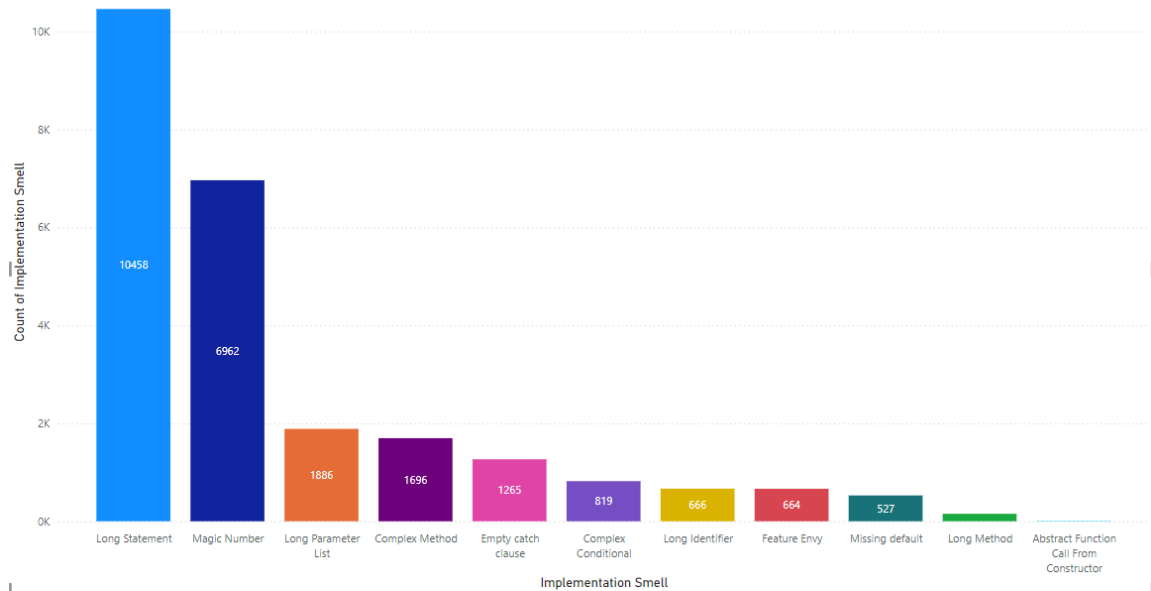


Figure 4.7: Code Smells Count

Until now, we have compared the survival of code smells versus test smells to determine which persists longer. Now, we aim to identify which specific test smells have a higher survival rate compared to code smells and vice versa.

4.1.4 Code Smells

Figure 4.7 gives the frequency of code smells we researched across all repositories and Figure 4.8 shows the survivability of each code smell across all repositories. From Figure 4.7 we can see that both Long Statement and Magic Number has a high frequency compared to the rest of the smells but the median survivability of both smells is similar to the rest of the smells indicating that the smells were not prioritised even with its unusually high frequency, but Empty Catch Clause smell has a high rate of refactoring done on it, also having one of the lowest median survivability indicating that this smell is prioritised more the rest of the smell. Most of the smells

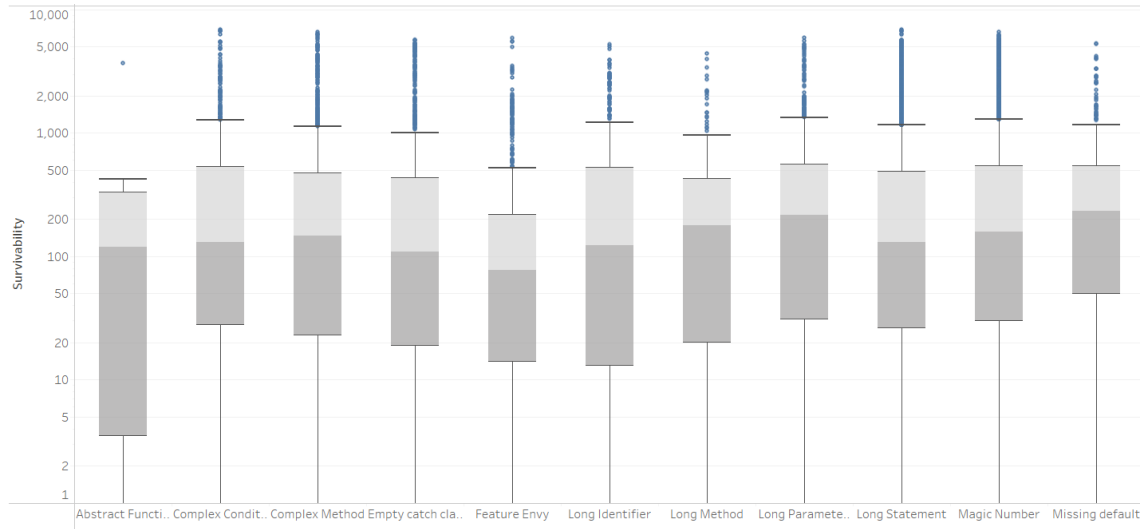


Figure 4.8: Code Smell Survivability

have a low frequency and those that were present were refactored around 150 days, but Missing Default and Long Parameter List has median survivability over 200 days and the majority of the smells are removed within 200 days.

Figure 4.9 give us information about code smell survival probability in days. The curve also considers the code smells that are still open at the time of mining and are represented as the dashes on the curve; as the survival curve considers open smells, the results obtained from them may differ from the results obtained from the box plots and the frequency charts above. Table 4.5 gives an overview of code smells removed and not removed in the repositories. On average 39% of code smells are not removed from the systems.

Code Smell	Not Removed	Removed	Not Removed (%)	Removed (%)
Long Statement	9182	10458	47	53
Magic Number	5792	6962	45	55
Long Parameter List	1105	1886	37	63
Complex Method	1001	1696	37	63
Empty catch clause	384	1265	23	77
Long Identifier	684	666	51	49
Complex Conditional	386	819	32	68
Missing default	593	527	53	47
Feature Envy	188	664	22	78
Long Method	67	153	30	70
Abstract Function call	7	8	47	53

Table 4.5: Code Smell Counts and Removal Percentages

- For the first 1000 days, the survival curves for all identified smells exhibit a steep decline up to 50%, indicating that a significant proportion of these smells

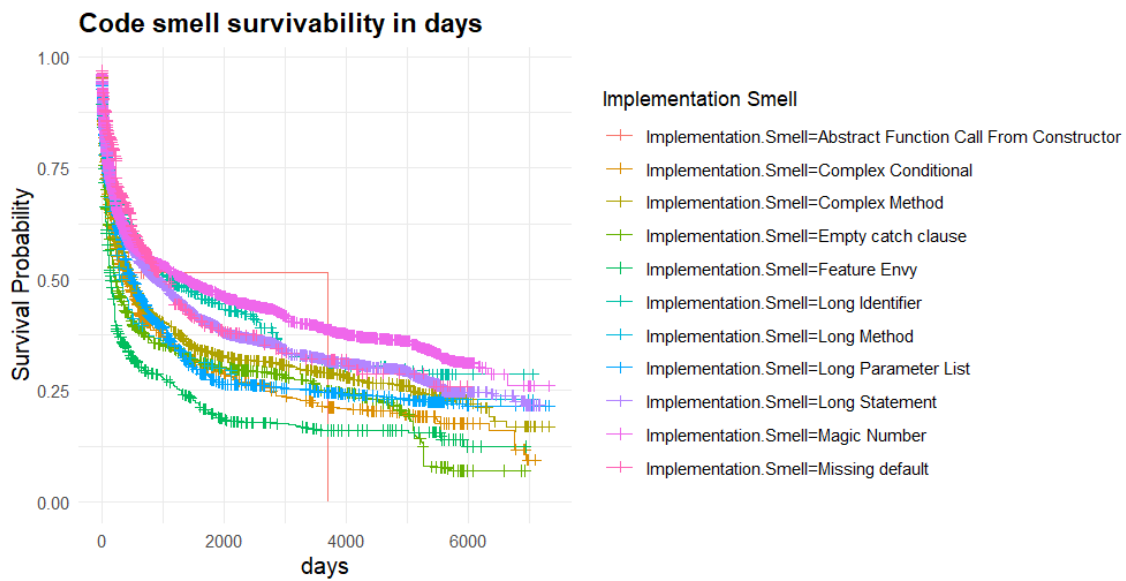


Figure 4.9: Code Smell Survival Curve

are removed relatively quickly after their introduction. This trend suggests that developers actively address many smells during the initial phase of their lifecycle.

- **Magic Number** has the highest survivability at any point in time compared to other smells.
- **Long Identifier**, **Long Statement**, **Missing Default**, and **Complex Number** follow in descending order of survival probability. Notably, their survival trends are relatively consistent across all repositories.
- Among the most frequently occurring smells, **Magic Number** and **Long Statement** also exhibit high survival probabilities, suggesting that these smells persist longer despite their frequency.
- **Empty Catch Clause**, in contrast, has low survivability, with most instances being refactored or resolved relatively quickly.
- **Feature Envy** and **Empty Catch Clause** shows the lowest survivability, indicating it is the most promptly addressed smell among those studied.
- The survival curve for **Long Identifier** (bluish-green color) shows a sudden dip around 2500 days, indicating a significant reduction in instances of this smell during this period. Similarly, the curve for **Empty Catch Clause** (light green color) exhibits a sharp decline around 5000 days, suggesting that many occurrences of this smell are addressed or refactored at this later stage in the software's lifecycle.

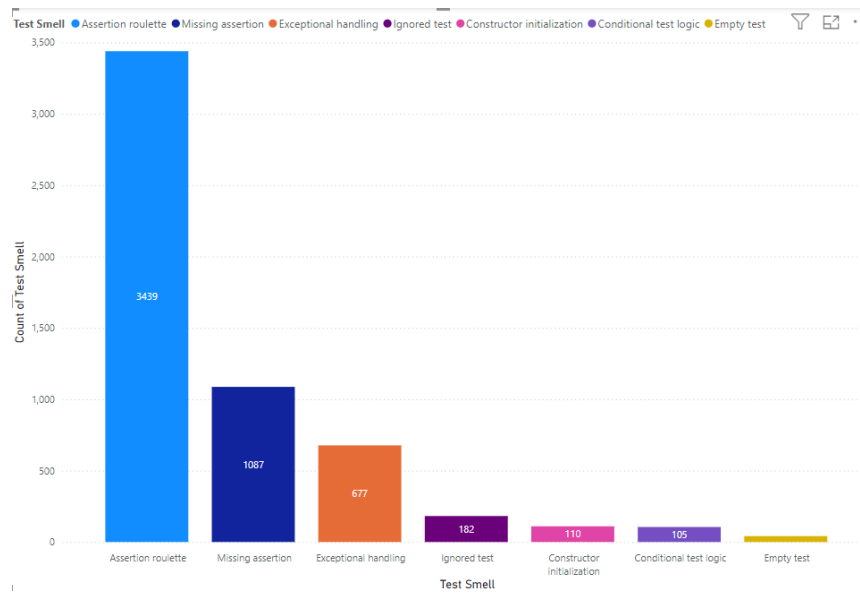


Figure 4.10: Test Smell Count

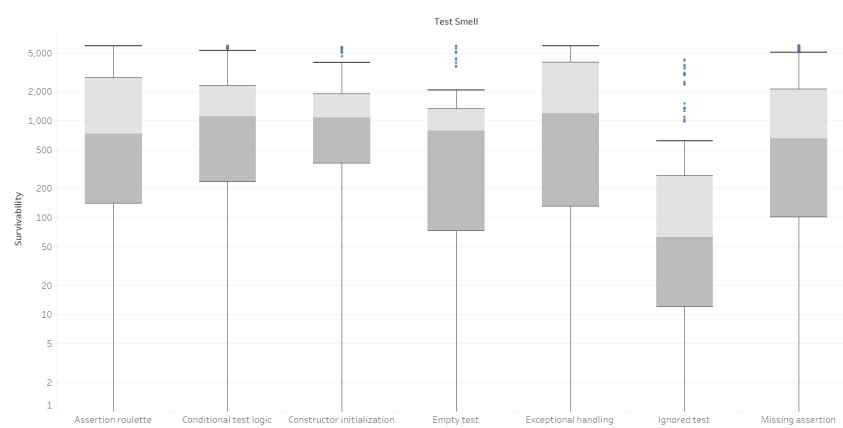


Figure 4.11: Test Smell Survivability

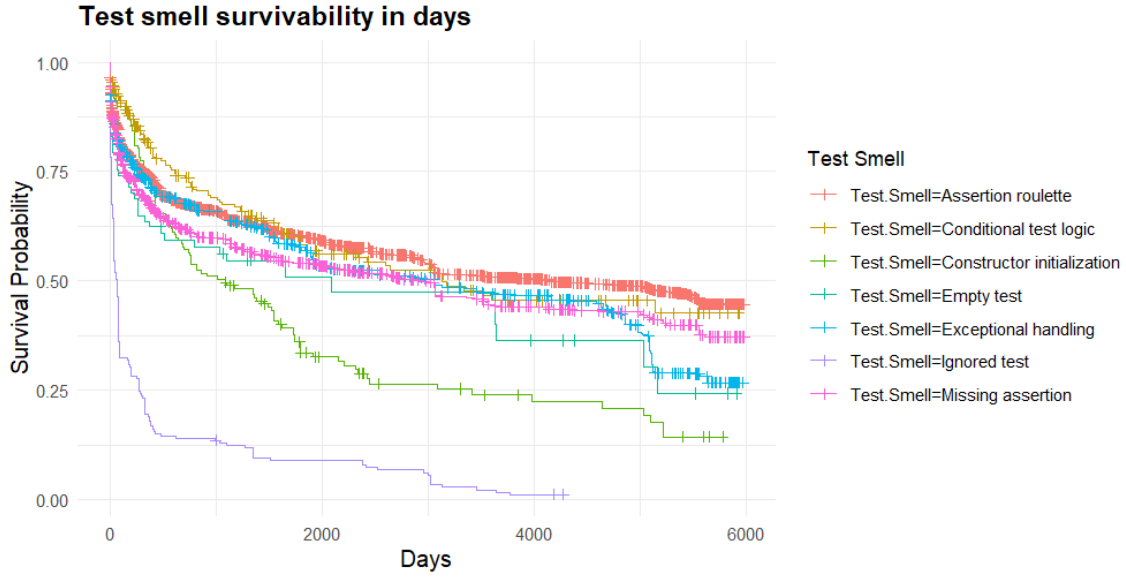


Figure 4.12: Test Smell Survival Curve

4.1.5 Test Smells

Figure 4.10 gives us the frequency of test smells we researched across all the repositories and Figure 4.11 shows the survivability of each test smell across all repositories. From Figure 4.10 we can see that Assertion Roulette and Missing Assertion has a very high frequency compared to the rest of the smells, it also has a low median survivability indicating that it may be prioritized. Exceptional Handling also has a high frequency compared to other smells but its survivability is also high, the rest of the smells have a low frequency and high median values with the exception of Ignored Test which has a low median value. Figure 4.12 give us the information about code

Test Smell	Not Removed	Removed	Not Removed (%)	Removed (%)
Assertion roulette	5003	3439	59.26	40.74
Missing assertion	1239	1087	53.27	46.73
Exceptional handling	569	677	45.67	54.33
Conditional test logic	157	105	59.92	40.08
Ignored test	3	182	1.62	98.38
Constructor initialization	37	110	25.17	74.83
Empty test	36	41	46.75	53.25

Table 4.6: Test Smell Counts and Removal Percentages

smells survival probability in days. These results may differ from the above observation as we also include not removed test smells. Table 4.6 gives an overview of test smells removed and not removed in the repositories. On average 42% of test smells are not removed from the systems.

- **Assertion roulette** shows the slowest rate of decay, maintaining a high survival probability even after many days. This suggests that developers are either

unaware of this issue, consider it less important, or find it particularly challenging to refactor.

- **Conditional test logic** and **Empty test** also show relatively high persistence, indicating they might be more difficult or less prioritized for refactoring.
- **Ignored test** seems to be addressed more quickly than others, as its survival probability drops more rapidly. There is 80% chance that it is removed within a 100 days.

4.1.6 Summary of RQ1:

Up to 39% of code smells and 42% of test smells remain unresolved in the systems during their evolution. When these smells are removed, code smells are typically resolved within 350 commits and 140 days, while test smells are resolved within 348 commits and 170 days. Additionally, approximately 40% of both code smells and test smells are removed by deleting the code affected by the smell. Test smells exhibit higher survivability than code smells, with a greater survival probability at any point in time. This suggests that test smells are more persistent in software systems, likely due to lower prioritisation during the development and maintenance process. Among code smells, Magic Number exhibits the highest survivability, with over 30% chance of affected instances persisting until the end of a project's lifecycle. Similarly, in the category of test smells, Assertion Roulette has the highest survivability, with over 50% chance of affected instances remaining unresolved by the project's lifecycle.

In contrast, Feature Envy has the lowest survivability among code smells, with only 12% chance of its instances persisting until the end of the project. Most Feature Envy occurrences are resolved within 80 days of their creation. Another notable code smell, Empty Catch Clause, is the most prioritised for resolution, with 77% of its occurrences being addressed promptly.

For test smells, Ignored Test demonstrates the least survivability, with nearly 0% chance of affected instances remaining unresolved by the end of the project. Approximately 98% of Ignored Test smells are resolved, making it the most prioritized test smell. Notably, most of these instances are removed within 60 days of their creation.

4.2 Association Rule Mining (RQ2)

Figure 4.13 presents the results of association rule mining, highlighting the co-occurrence of code smells and test smells. The table is represented in the form of a heat map where the low values are depicted using the colour red, the high values are represented in green and the moderate values in shades of yellow. The columns in the table represent the following values:

- **Support:** Indicates how frequently a rule appears in the dataset.
- **Confidence:** Measures the likelihood that the conclusion (right-hand side) of the rules is true when the condition (left-hand side) is met.

ID	Rules	support	confidence	lift	count
1	{Empty test} $\Rightarrow$ {Long Statement}	0.013518	0.817146	1.077957	12620
2	{Conditional test logic} $\Rightarrow$ {Long Statement}	0.034	0.842857	1.111874	31742
3	{Exceptional handling} $\Rightarrow$ {Long Statement}	0.190941	0.811762	1.070855	178263
4	{Assertion roulette} $\Rightarrow$ {Long Statement}	0.681427	0.762185	1.005453	636181
5	{Empty test, Missing assertion} $\Rightarrow$ {Long Statement}	0.013518	0.817146	1.077957	12620
6	{Conditional test logic, Exceptional handling} $\Rightarrow$ {Long Parameter List}	0.012602	0.583437	3.658238	11765
7	{Conditional test logic, Exceptional handling} $\Rightarrow$ {Complex Method}	0.013293	0.615423	1.918658	12410
8	{Conditional test logic, Exceptional handling} $\Rightarrow$ {Long Statement}	0.021599	1	1.319173	20165
9	{Assertion roulette, Conditional test logic} $\Rightarrow$ {Long Statement}	0.034	0.842857	1.111874	31742
10	{Exceptional handling, Missing assertion} $\Rightarrow$ {Long Statement}	0.060525	0.772605	1.019199	56506
11	{Assertion roulette, Exceptional handling} $\Rightarrow$ {Long Statement}	0.181394	0.809195	1.067468	169350
12	{Assertion roulette, Conditional test logic, Exceptional handling} $\Rightarrow$ {Long Parameter List}	0.012602	0.583437	3.658238	11765
13	{Assertion roulette, Conditional test logic, Exceptional handling} $\Rightarrow$ {Complex Method}	0.013293	0.615423	1.918658	12410
14	{Assertion roulette, Conditional test logic, Exceptional handling} $\Rightarrow$ {Long Statement}	0.021599	1	1.319173	20165
15	{Assertion roulette, Exceptional handling, Missing assertion} $\Rightarrow$ {Long Statement}	0.056158	0.773141	1.019906	52429

Figure 4.13: Rule Mining Results

- **Lift:** Determines the strength of the association between LHS and RHS. A lift value greater than 1 indicates a positive correlation, and less than 1 indicates a negative correlation.
- **Count:** The absolute frequency of the rule's occurrence in the dataset.

We also performed a chi-square test to see whether the rules are statistically significant and found that all rules are statistically significant.

4.2.1 Smells with High Support Value

- *Assertion Roulette* $\rightarrow$ *Long Statement* has the highest support value:
 - Support: 0.68, Confidence: 0.76, Count: 636,181
- This indicates many instances of this association in the dataset, suggesting that whenever "Assertion Roulette" appears in the code, it is often followed by or associated with a "Long Statement".
- **Long Statement** and **Assertion Roulette** are among the most frequent and enduring code smells, which could explain why they have a higher probability of co-occurrence. Their persistent presence in the codebase increases the likelihood of their association, contributing to their higher support and confidence values.

4.2.2 Smells with High Confidence

- *Conditional Test Logic, Exceptional Handling* $\rightarrow$ *Long Statement*
- *Assertion roulette, Conditional Test Logic, Exceptional Handling* $\rightarrow$ *Long Statement*
- Both these rules have the same support(0.021), confidence(1) and lift(1.3) values.

- As seen from the definition of the smells, Exception Handling and Conditional Test Logic arise when the code has many exceptions and complex conditions. These can result from the presence of Long Statements, as they occur when the logic of the code becomes overly detailed and convoluted. Long Statements often emerge when developers try to handle multiple scenarios or edge cases within a single line or block of code, which may lead to complex conditional branches or extensive exception handling, and Assertion Roulette's presence can be due to its high frequency.

4.2.3 Compound Smells with High Correlation

Conditional Test Logic + Exceptional Handling:

- This combination frequently leads to other smells such as **Long Parameter List**, **Complex Method**, and **Long Statement**.
- Example:
 - *Conditional Test Logic, Exceptional Handling* $\rightarrow$ *Long Parameter List*:
 - * Lift: 3.658238
 - The same combination leads to Complex Method with a lift of 1.918658 and to Long Statement with perfect confidence (1.0), suggesting this pairing is persistent and often associated with multiple smells.
- Complex production code, characterised by Long Parameter Lists, Long Methods, and Complex Methods, often leads to equally complex test code. As the production code becomes more intricate, test cases must account for these complexities and may introduce Conditional Test Logic and Exception Handling. For example, a method with a Long Parameter List may require tests with many inputs, increasing the complexity of the test logic and potentially involving multiple conditions or exceptions.

4.2.4 Summary of RQ2

The "Long Statement" code smell is prominently linked to various test smells such as "Assertion Roulette", "Conditional Test Logic", and "Exceptional Handling", both individually and in combinations. Similarly, specific combinations of test smells (e.g., "Conditional Test Logic" and "Exceptional Handling") are strongly associated with "Long Parameter List" and "Complex Method" code smells. The smells associated with the Long Parameter List have high lift values indicating high correlation.

This study aimed to investigate the survivability of code smells and test smells within software systems. To achieve this, Firstly, we performed survival analysis to determine the persistence of code smells and test smells over time, allowing us to compare their respective survivability rates. Secondly, association rule mining techniques were utilised to identify potential relationships between specific types of code smells and the presence of associated test smells. Combining these analytical methods, this study sought deeper insight into the interplay between code and test quality, ultimately contributing to a better understanding of software evolution and maintenance challenges.

5.1 What Smells tend to survive more in Software projects

Our survival analysis found that 39% of code smells and 42% of test smells remain unresolved during system evolution. In contrast, Tufano’s study reported that approximately 80% of code smells [51] and 98% of test smells [50] remain unresolved. This significant difference could be attributed to the scope of the research. Tufano’s study analyzed over 200 open-source projects from ecosystems such as Apache, Android, and Eclipse, including both active and inactive repositories. In contrast, our study focused solely on actively contributed projects. The inclusion of less actively maintained projects in Tufano’s study may have led to the higher percentage of unresolved smells, particularly the 98% of test smells reported. When these smells are removed, 50% of code smells are resolved within approximately 140 days and 350 commits, whereas test smells typically take around 170 days and 348 commits to resolve. This finding, as illustrated in Figures 4.1 and 4.2, highlights that while test smells tend to persist longer than code smells, the effort required to resolve them, measured by the number of commits, remains nearly identical. This suggests that the longer survival time of test smells is less likely due to increased complexity or difficulty of resolution and more plausibly attributable to their lower prioritisation within the development process. Overall, test smells exhibit a higher survival rate compared to code smells.

However, a repository-specific analysis reveals notable differences in the survivability patterns of smells. As shown in Figure 4.3, certain repositories, such as Mongo-java-driver and SeaTunnel, exhibit higher survivability for code smells com-

pared to test smells. This observation holds true for closed intervals (where smells are removed from the system); however, some differences emerge in the results for open intervals (which include smells that are not resolved). These differences can be attributed to the percentage of unresolved smells. Only the Mongo-java-driver repository demonstrates a higher code smell survival rate than test smell survival, as illustrated in Figure 4.5. Further examination of Table 4.1, indicated that these repositories are characterised by significantly larger project sizes than others. The scale and complexity inherent to larger projects may likely contribute to the delayed resolution of code smells relative to smaller systems [41]. This finding diverges from prior research by Habchi [19], that code smells are resolved more quickly in larger systems. However, this discrepancy appears specific to the Android ecosystem, as evidenced by Tufano [51], who demonstrated that Android systems exhibit lower smell survival rates than other ecosystems. These variations highlight the significance of contextual factors, such as project size and ecosystem characteristics, in influencing the survival patterns of code and test smells. Also, 47% of code smells in the study were removed by deleting the files, and 45% of test smells were removed by deleting files. This is shown in Table 4.2. This also aligns with a study by Tufano [51] that 40% of code smells are removed from code deletion, we considered the smell is removed by code deletion if the file is deleted in the smell removing commit. Regarding commits, the survival probability code smells are always less than the test smell at the same commit. This can be observed from Figure 4.4 and Table 4.4. This suggests that code smells are more prioritised over test smells. For comparing the survival curves of code smells and test smells, we took 500 days and 500 commits as a reference point, we observed that

For 500 Days:

- Code Smell : has chance of 54% to survive until 500 days.
- Test Smell : has chance of 67% to survive until 500 days.

For 500 Commits:

- Code Smell : has chance of 64% to survive until 500 commits.
- Test Smell : has chance of 72% to survive until 500 commits.

Similar results for test smell survivability were observed in Tufano’s study [50], where test smells persisted for up to 1800 days and 1000 commits. This difference may stem from the types of smells analyzed in their study compared to ours. For code smells, Tufano’s findings [51] showed survival times ranging from 800 to 2000 days and between 500 and 2000 commits, depending on the ecosystem under analysis, this is due to considering wide range of code smells than previous studies. The overall survival patterns of code and test smells provide valuable insights, but examining the survivability of specific smells is crucial. Our findings reveal that Magic Number has the highest survivability, which aligns with the research by Mahbub et al. (2024) [25]. This trend is also evident in Figure 4.9, where Magic Number shows prolonged persistence, and it is one of the most frequently occurring code smells. In contrast, Feature Envy has the lowest survival probability and exhibits the highest

removal percentage, as seen in Table 4.5. This suggests that Feature Envy is a highly prioritised code smell, a finding that aligns with the research by Peters [38].

In Test Smells Association Roulette has the highest survival probability followed by Conditional Test Logic. Assertion Roulette is also one of the most common test smells, and it appears to have most not removed instances, indicating that they are of low priority; from [4], it is said that Assertion Roulette has minimal impact on maintainability, so they are least prioritised, resulting in high survivability. Despite having a low frequency compared to the rest of the test smells, Ignored Tests have over 90% of instances removed, indicating that the smell is prioritised more than high-frequency smells like Assertion Roulette and Missing Assertion. Both smells have high frequency and survivability, which aligns with the research of Palombo [33], but in his research, the Eager Test is also one of the most common smells, but in our research, we haven't found a single instance of it, despite our tool being capable of identifying it [52]. This discrepancy could be due to the nature of the code analysed in our research, as it is not generated, whereas Eager Test is more frequently observed in generated test code.

5.2 Are certain Test Smells more likely to associate with particular Code Smells

Based on the results of the association rule mining, the analysis provides evidence that specific test smells such as Empty Test, Conditional Test Logic, Exceptional Handling, and Assertion Roulette consistently correlate with code smells such as Long Statement, Long Parameter List, and Complex Method. These findings suggest that production code quality, directly or indirectly, impacts the quality of test code, often leading to compounding test smells.

Assertion Roulette is often associated with Long Statements, as evidenced in Table 4.13. The results show a high support value of 0.68 and a confidence value of 0.76 for the association between these two smells, indicating a strong and frequent relationship. This suggests that whenever Assertion Roulette is present in the test code, there is a high likelihood that it is linked to a Long Statement in the production code. The high support value signifies that this pattern occurs in a significant portion of the analyzed dataset, making it a recurring and notable issue.

Additionally, Assertion Roulette and Long Statements are the most frequent test smells and code smells, respectively, in the analyzed repositories, as shown in Figures 4.7 and 4.10. This high frequency further explains their strong association. Long Statements often involve large and complex logic, and tests written for such methods typically include multiple assertions to validate various aspects of the logic. Without clear documentation or proper structure, these assertions result in Assertion Roulette, as noted in [53].

From Table 4.13 we observe that out of the 15 rules present Long Statement is the most reoccurring one on RHS, this can be due to its high frequency of the smell in the repositories compared to other code smells this can be seen in Table 4.5. Long Statement is often associated with Empty Test, Conditional Test logic, and Exceptional Handling, this can be explained by observing their definitions:

- **Empty Test and Long Statement:** An Empty Test means that the test is incomplete or doesn't properly validate the production code. This often happens when the production method is very long and complex, making it harder for developers to write good tests. As a result, they may leave the test incomplete, which is why Empty Test often occurs alongside Long Statement.
- **Conditional Test Logic and Long Statement:** Conditional Test Logic happens when tests include if statements or other branches to handle different cases [26]. This is usually needed when the production method is complicated, like a Long Statement. The long length of the production code forces developers to write more complex test logic, which explains their frequent association.
- **Exceptional Handling and Long Statement:** Exceptional Handling appears in tests when production methods rely a lot on exceptions. Since Long Statements often include many conditions and exceptions, tests for such methods need detailed logic to handle all possible failures. This is why tests for long statements often include excessive exception handling, leading to this smell.

Whenever Conditional Test Logic and Exceptional Handling are present, there is a high confidence that these issues are associated with Long Statement in the production code. The confidence value is consistently 1 whenever these test smells occur. From this, we can infer that Conditional Test Logic and Exceptional Handling may be caused by the presence of Long Statements in the production code. Also, we observed from Figures 4.9 and 4.12 that there is a dip around 5000 days in Long Statement, Conditional Test Logic, Empty Test and Exceptional Handling. This may indicate there is some relationship between them.

5.3 Threats to Validity

5.3.1 Construct Validity

The term "construct validity" refers to ensuring that the methods survival analysis and association rule mining used in a thesis accurately represent the principles of finding survivability's for code and test smells, and finding any associations between code smells and test smells. One threat to construct validity is the inaccuracy of the tool to detect smells accurately causing deviations and incorrect interpretation of the results. To mitigate this we have selected a DesigniteJava [44, 52] that has been evaluated previously having high precision and accuracy making it a very good choice to identify the smells.

Another threat is the tool's ability to gather complete commit data. In some cases, DesigniteJava does not analyze all commits, leading to potential data omissions. To address this, we cross-checked the analyzed commits against git log data. If any commits were omitted, we manually performed mining on those specific commits to ensure completeness.

5.3.2 Conclusion Validity

Conclusion validity refers to the reliability of the conclusions drawn from a study. In our research, it pertains to whether the associations identified between code smells and test smells are genuine. To mitigate potential biases, we employed association rule mining to uncover these relationships which is commonly used to find relations [50] and conducted a chi-square test to confirm whether the associations are statistically significant.

5.3.3 External Validity

External validity refers to the extent to which the findings of a study can be applied to different contexts. One major concern in this study is that the data is collected from repositories of a single language(java). While the findings we obtain may be valid for the java systems, but they may not be applicable for systems written in other languages. Developments in the field could also pose a threat to the results since they could reveal completely new smells. As more smells are introduced, our findings might become outdated unless they are once again researched taking the new smells into consideration.

5.4 Limitations

While our research provides valuable insights into the relationships between test smells and code smells, as well as their survivability patterns, several limitations must be acknowledged:

- **Dataset** : Due to computational limits of our system we have only mined 5 repositories mining more repositories of various code bases and languages and contributors can give a clear answer on how the survivability trend is and also eliminate any bias present.
- **Smells** : Our thesis focuses on smells presented by the tool DesigniteJava, apart for the smell provided by DesigniteJava there are many more test and code smells present, considering these smells may change the survivability orders and also may introduce new associations between code smells and test smells.
- **Repository Size**: In our thesis we used only Repositories size up to 600Kloc due to Time taking processing of mining them for smells, but our study unexpectedly revealed that Size of the Repository has impact on survivability of code and test smells. So there is a need to find the dependence of survivability of test smells and code smells with other factors like Size of the Repository, # Contributors etc
- **Generalization**: This study performed survival analysis and association rule mining exclusively on Java projects. While the findings are valid for Java-based systems, they may not generalize to other programming languages or application types. Different ecosystems, such as web applications or embedded systems, may exhibit distinct survivability patterns for code and test smells.

6.1 Conclusion

The aim of this thesis was to learn whether code smell or test smell survived more in a project and also to find any association between them. The findings of the study show that test smells survive more than code smells in a project, and the Smell Long Statement has a high chance of appearing in a production code file, which is associated with a test file containing the smells Conditional Test Logic or Exceptional Handling.

The study addresses the survivability of both code smells and test smells in the same project, an area overlooked in previous research. By learning which smell survives more we can understand which type of smell is being prioritized by the developers and can help in creation of new prioritization techniques. This research also studies the association of code smells and test smell between production code and its associated test code respectively, bridging a critical gap in the field.

This thesis shows that test smells survive more than code smells likely due to prioritization of developers, however when specific smells were observed for test smell Ignored Test more than 90% of the instances were removed indicating targeted effort from the developers to remove it compared to the rest of test smell where only 50% of them were removed whereas in code smell around 65% were removed. We also observed that survivability of code smell in small projects ranging 100-200KLOC has small survivability compared to larger projects ranging 200KLOC and above.

One major finding is that the code smell Long Statement has a probability of 80% to appear in production code which is associated with a test code containing either Conditional Test Logic or Exceptional Handling Smell is present, but when both the test smell occur at the same time the probability of Long Statement appearing is 100%.

This research investigates the survivability of test smells and code smells, as well as their potential associations in software systems. The findings contribute to understanding the persistence of these smells within repositories, offering insights into prioritization trends. Specifically, this study reveals which smells are addressed more frequently and which are un-prioritised, shedding light on the decision-making processes during software maintenance.

Additionally, the identification of associations between code smells and test smells provides valuable information for improving strategies to prevent or resolve these issues. By leveraging these results, researchers and practitioners can refine static analysis tools to enhance the detection of code and test smells. Furthermore, the

insights gained can guide the development of more effective techniques for mitigating the impact of smells, ultimately leading to better software quality and maintainability. Our results can help developers leverage the identified associations to detect and refactor test smells early based on the presence of related code smells.

6.2 Future Work

Future work is proposed based on the limitations and unexpected results encountered in this thesis. One significant finding is that factors such as project size influence the survivability of code smells and test smells. Specifically, the study observed that test smells tend to survive longer than code smells in projects with less than 600KLOC, while code smells survive longer in projects exceeding this size. However, since only one project in the dataset exceeds 600KLOC, this conclusion lacks strong statistical support. To address this limitation, future studies should examine projects of varying sizes to validate and generalize these findings.

This study focused exclusively on Java-based projects, which limits the applicability of the results to other programming languages. Expanding this research to include projects developed in languages such as Python, C#, or JavaScript would provide broader insights into the survivability of code smells and test smells across diverse software ecosystems. Additionally, there is a need to investigate the reasons behind the removal of code smells and test smells. While we were able to identify smells that were deleted, we did not find sufficient information on other possible reasons for their removal. Understanding whether smells are removed due to refactoring, code restructuring, requirement changes, or other factors would provide deeper insights into software maintenance practices and the evolution of code quality.

Future work could also investigate other factors influencing smell survivability, such as team size, development methodologies, or the use of specific tools or frameworks. Moreover, exploring the impact of smell refactoring on software quality and its correlation with survivability trends could yield valuable insights for practitioners aiming to maintain high-quality codebases. Our research identified associations between code smells and test smells, focusing primarily on the presence of these smells within software projects. However, this study did not explore whether the removal of specific code smells influences the removal of associated test smells. Future work could investigate this dynamic relationship to determine if addressing certain code smells has a cascading effect on test smell removal. This could help in prioritizing refactoring efforts by identifying high-impact smells whose removal might improve both production and test code quality.

Additionally, future studies could analyze the temporal co-evolution of code smells and test smells to understand whether they tend to be introduced and resolved together during the software lifecycle. Exploring the role of different development practices, such as test-driven development or continuous integration, in influencing these associations could also provide valuable insights. Another avenue of research could involve examining how these associations vary across programming languages and frameworks, offering a broader perspective on smell management. Finally, leveraging machine learning techniques to predict the likelihood of smell associations and their persistence could lead to more effective tools for maintaining and improving

software quality.

References

- [1] M. Abbes, F. Khomh, Y.-G. Gueheneuc, and G. Antoniol, “An empirical study of the impact of two antipatterns, blob and spaghetti code, on program comprehension,” in *2011 15Th european conference on software maintenance and reengineering*. IEEE, 2011, pp. 181–190.
- [2] W. Aljedaani, A. Peruma, A. Aljohani, M. Alotaibi, M. W. Mkaouer, A. Ouni, C. D. Newman, A. Ghallab, and S. Ludi, “Test smell detection tools: A systematic mapping study,” in *Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering*, 2021, pp. 170–180.
- [3] S. Amann, S. Beyer, K. Kevic, and H. Gall, “Software mining studies: Goals, approaches, artifacts, and replicability,” *Software Engineering: International Summer Schools, LASER 2013-2014, Elba, Italy, Revised Tutorial Lectures 10*, pp. 121–158, 2015.
- [4] G. R. Bai, K. Presler-Marshall, S. R. Fisk, and K. T. Stolee, “Is assertion roulette still a test smell? an experiment from the perspective of testing education,” in *2022 IEEE Symposium on visual languages and human-centric computing (VL/HCC)*. IEEE, 2022, pp. 1–7.
- [5] G. Bavota, A. Qusef, R. Oliveto, A. De Lucia, and D. Binkley, “An empirical analysis of the distribution of unit test smells and their impact on software maintenance,” in *2012 28th IEEE International Conference on Software Maintenance (ICSM)*, 2012, pp. 56–65.
- [6] G. Bavota, A. Qusef, R. Oliveto, A. Lucia, and D. Binkley, “Are test smells really harmful? an empirical study,” *Empirical Softw. Engg.*, vol. 20, no. 4, p. 1052–1094, aug 2015. [Online]. Available: <https://doi.org/10.1007/s10664-014-9313-0>
- [7] M. Breugelmans and B. Van Rompaey, “Testq: Exploring structural and maintenance characteristics of unit test suites,” in *WASDeTT-1: 1st International Workshop on Advanced Software Development Tools and Techniques*. Citeseer, 2008, p. 11.
- [8] D. Campos, L. Martins, C. Bezerra, and I. Machado, “Investigating developers’ contributions to test smell survivability: A study of open-source projects,” in *Proceedings of the 8th Brazilian Symposium on Systematic and Automated Software Testing*, 2023, pp. 86–95.
- [9] P. Chatterjee, “Extracting archival-quality information from software-related chats,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*, 2020, pp. 234–237.

- [10] Z. Chen, L. Chen, W. Ma, and B. Xu, “Detecting code smells in python programs,” in *2016 international conference on Software Analysis, Testing and Evolution (SATE)*. IEEE, 2016, pp. 18–23.
- [11] O. Dabic, E. Aghajani, and G. Bavota, “Sampling projects in github for msr studies,” in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 2021, pp. 560–564.
- [12] M. De Stefano, F. Pecorelli, D. Di Nucci, and A. De Lucia, “A preliminary evaluation on the relationship among architectural and test smells,” in *2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2022, pp. 66–70.
- [13] E. Fernandes, J. Oliveira, G. Vale, T. Paiva, and E. Figueiredo, “A review-based comparative study of bad smell detection tools,” in *Proceedings of the 20th international conference on evaluation and assessment in software engineering*, 2016, pp. 1–12.
- [14] F. A. Fontana, V. Ferme, and M. Zanoni, “Towards assessing software architecture quality by exploiting code smell relations,” in *2015 IEEE/ACM 2nd International Workshop on Software Architecture and Metrics*, 2015, pp. 1–7.
- [15] F. A. Fontana, V. Ferme, M. Zanoni, and A. Yamashita, “Automatic metric thresholds derivation for code smell detection,” in *2015 IEEE/ACM 6th International Workshop on Emerging Trends in Software Metrics*. IEEE, 2015, pp. 44–53.
- [16] M. Fowler, *Refactoring: improving the design of existing code*. Addison-Wesley Professional, 2018.
- [17] Y. Fushihara, H. Aman, S. Amasaki, T. Yokogawa, and M. Kawahara, “A trend analysis of test smells in python test code over commit history,” in *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 2023, pp. 310–314.
- [18] A. Garg, M. Gupta, G. Bansal, B. Mishra, and V. Bajpai, “Do bad smells follow some pattern?” in *Proceedings of the International Congress on Information and Communication Technology: ICICT 2015, Volume 1*. Springer, 2016, pp. 39–46.
- [19] S. Habchi, R. Rouvoy, and N. Moha, “On the survival of android code smells in the wild,” in *2019 IEEE/ACM 6th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. IEEE, 2019, pp. 87–98.
- [20] F. Khomh, M. D. Penta, Y.-G. Guéhéneuc, and G. Antoniol, “An exploratory study of the impact of antipatterns on class change-and fault-proneness,” *Empirical Software Engineering*, vol. 17, pp. 243–275, 2012.
- [21] S. Lambiase, A. Cupito, F. Pecorelli, A. De Lucia, and F. Palomba, “Just-in-time test smell detection and refactoring: The darts project,” in *Proceedings of the 28th international conference on program comprehension*, 2020, pp. 441–445.
- [22] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, “Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the

- median,” *Journal of experimental social psychology*, vol. 49, no. 4, pp. 764–766, 2013.
- [23] A. Lozano, K. Mens, and J. Portugal, “Analyzing code evolution to uncover relations,” in *2015 IEEE 2Nd international workshop on patterns promotion and anti-patterns prevention (PPAP)*. IEEE, 2015, pp. 1–4.
- [24] K. Lucas, R. Gheyi, E. Soares, M. Ribeiro, and I. Machado, “Evaluating large language models in detecting test smells,” *arXiv preprint arXiv:2407.19261*, 2024.
- [25] R. Mahbub, M. M. Rahman, and M. A. Habib, “On the prevalence, evolution, and impact of code smells in simulation modelling software,” in *2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2024, pp. 154–165.
- [26] F. Maier and M. Felderer, “Detection of test smells with basic language analysis methods and its evaluation,” in *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2023, pp. 897–904.
- [27] R. Marinescu, “Detection strategies: Metrics-based rules for detecting design flaws,” in *20th IEEE International Conference on Software Maintenance, 2004. Proceedings.* IEEE, 2004, pp. 350–359.
- [28] L. Martins, H. Costa, and I. Machado, “On the diffusion of test smells and their relationship with test code quality of java projects,” *Journal of Software: Evolution and Process*, p. e2532, 2023.
- [29] G. Meszaros, *xUnit test patterns: Refactoring test code*. Pearson Education, 2007.
- [30] N. Moha, Y.-G. Guéhéneuc, L. Duchien, and A.-F. Le Meur, “Decor: A method for the specification and detection of code and design smells,” *IEEE Transactions on Software Engineering*, vol. 36, no. 1, pp. 20–36, 2009.
- [31] F. Palomba, G. Bavota, M. Di Penta, F. Fasano, R. Oliveto, and A. De Lucia, “A large-scale empirical study on the lifecycle of code smell co-occurrences,” *Information and Software Technology*, vol. 99, pp. 1–10, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584918300211>
- [32] F. Palomba, G. Bavota, M. Di Penta, F. Fasano, R. Oliveto, and A. De Lucia, “On the diffuseness and the impact on maintainability of code smells: a large scale empirical investigation,” in *Proceedings of the 40th International Conference on Software Engineering*, 2018, pp. 482–482.
- [33] F. Palomba, D. Di Nucci, A. Panichella, R. Oliveto, and A. De Lucia, “On the diffusion of test smells in automatically generated test code: An empirical study,” in *Proceedings of the 9th international workshop on search-based software testing*, 2016, pp. 5–14.
- [34] F. Palomba, R. Oliveto, and A. De Lucia, “Investigating code smell co-occurrences using association rule learning: A replicated study,” in *2017 IEEE Workshop on Machine Learning Techniques for Software Quality Evaluation (MaLTeSQuE)*. IEEE, 2017, pp. 8–13.

- [35] F. Palomba, A. Panichella, A. De Lucia, R. Oliveto, and A. Zaidman, “A textual-based technique for smell detection,” in *2016 IEEE 24th international conference on program comprehension (ICPC)*. IEEE, 2016, pp. 1–10.
- [36] A. Peruma, K. Almalki, C. D. Newman, M. W. Mkaouer, A. Ouni, and F. Palomba, “Tsdetect: An open source test smells detection tool,” in *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2020, pp. 1650–1654.
- [37] A. Peruma, K. S. Almalki, C. D. Newman, M. W. Mkaouer, A. Ouni, and F. Palomba, “On the distribution of test smells in open source android applications: An exploratory study,” 2019.
- [38] R. Peters and A. Zaidman, “Evaluating the lifespan of code smells using software repository mining,” in *2012 16th European conference on software maintenance and reengineering*. IEEE, 2012, pp. 411–416.
- [39] A. Qusef, M. O. Elish, and D. Binkley, “An exploratory study of the relationship between software test smells and fault-proneness,” *IEEE Access*, vol. 7, pp. 139 526–139 536, 2019.
- [40] S. Reichhart, T. Gîrba, and S. Ducasse, “Rule-based assessment of test quality.” *J. Object Technol.*, vol. 6, no. 9, pp. 231–251, 2007.
- [41] A. Rio and F. B. e Abreu, “Php code smells in web apps: Evolution, survival and anomalies,” *Journal of Systems and Software*, vol. 200, p. 111644, 2023.
- [42] T. Sharma, “Designite: a customizable tool for smell mining in c# repositories,” in *10th Seminar on Advanced Techniques and Tools for Software Evolution, Madrid, Spain*, 2017.
- [43] T. Sharma, M. Fragkoulis, and D. Spinellis, “Does your configuration code smell?” in *Proceedings of the 13th international conference on mining software repositories*, 2016, pp. 189–200.
- [44] T. Sharma, P. Mishra, and R. Tiwari, “Designite: A software design quality assessment tool,” in *Proceedings of the 1st International Workshop on Bringing Architectural Design Thinking into Developers’ Daily Activities*, 2016, pp. 1–4.
- [45] B. Sotto-Mayor, A. Mishali, M. Kalech, and R. Abreu, “Exploring designite for smell-based defect prediction.”
- [46] D. Spadini, F. Palomba, A. Zaidman, M. Bruntink, and A. Bacchelli, “On the relation of test smells to software code quality,” in *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2018, pp. 1–12.
- [47] G. Suryanarayana, G. Samarthiyam, and T. Sharma, *Refactoring for software design smells: managing technical debt*. Morgan Kaufmann, 2014.
- [48] A. Tahir, S. Counsell, and S. G. MacDonell, “An empirical study into the relationship between class features and test smells,” in *2016 23rd Asia-Pacific Software Engineering Conference (APSEC)*, 2016, pp. 137–144.
- [49] G. Travassos, F. Shull, M. Fredericks, and V. R. Basili, “Detecting defects in object-oriented designs: using reading techniques to increase software quality,” *ACM sigplan notices*, vol. 34, no. 10, pp. 47–56, 1999.

- [50] M. Tufano, F. Palomba, G. Bavota, M. Di Penta, R. Oliveto, A. De Lucia, and Poshyvanyk, “An empirical investigation into the nature of test smells,” in *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*, 2016, pp. 4–15.
- [51] M. Tufano, F. Palomba, G. Bavota, R. Oliveto, M. Di Penta, A. De Lucia, and D. Poshyvanyk, “When and why your code starts to smell bad (and whether the smells go away),” *IEEE Transactions on Software Engineering*, vol. 43, no. 11, pp. 1063–1088, 2017.
- [52] Tushar, “Multi-faceted code smell detection at scale using designitejava 2.0,” in *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, 2024, pp. 284–288.
- [53] A. Van Deursen, L. Moonen, A. Van Den Bergh, and G. Kok, “Refactoring test code,” in *Proceedings of the 2nd international conference on extreme programming and flexible processes in software engineering (XP2001)*. Citeseer, 2001, pp. 92–95.
- [54] B. Van Rompaey, B. Du Bois, S. Demeyer, and M. Rieger, “On the detection of test smells: A metrics-based approach for general fixture and eager test,” *IEEE Transactions on Software Engineering*, vol. 33, no. 12, pp. 800–817, 2007.
- [55] M. J. Ventresca and J. W. Mohr, “Archival research methods,” *The Blackwell companion to organizations*, pp. 805–828, 2017.
- [56] A. Yamashita, “How good are code smells for evaluating software maintainability? results from a comparative case study,” in *2013 IEEE International Conference on Software Maintenance*, 2013, pp. 566–571.
- [57] A. Yamashita and L. Moonen, “Do code smells reflect important maintainability aspects?” in *2012 28th IEEE international conference on software maintenance (ICSM)*. IEEE, 2012, pp. 306–315.
- [58] A. Yamashita, leon and Moonen, “Exploring the impact of inter-smell relations on software maintainability: An empirical study,” in *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 2013, pp. 682–691.
- [59] E. Zabardast, K. Ebo Bennin, and J. Gonzalez-Huerta, “Further investigation of the survivability of code technical debt items,” *Journal of Software: Evolution and Process*, vol. 34, no. 2, p. e2425, 2022.

